

Các thuật toán tối ưu hóa hàm tổn thất trong bài toán Hồi quy Logistic

Bộ dữ liệu ứng dụng ngân hàng bán lẻ (ẩn danh)

Học viên 1 (HV1) Học viên 2 (HV2)

MAT6202 — Tối ưu hóa nâng cao
Trường ĐH Khoa học Tự nhiên, ĐHQGHN

August 13, 2026

Nội dung trình bày

STT	Nội dung	Trang
1	Giới thiệu bài toán, hàm tổn thất và tính lồi	3
2	Lý thuyết các thuật toán: GD, AGD, Newton, SGD	7
3	Thiết kế thí nghiệm	15
4	Kết quả : dò tham số bằng thử và sai	19
5	Hằng số Lipschitz và chọn bước theo lý thuyết	38
6	Mở rộng: họ thuật toán Ada và Adam	44
7	Mở rộng: bài toán Lasso (lồi, không khả vi)	52
8	Kết luận	59

1. GIỚI THIỆU BÀI TOÁN

1.1. Phát biểu bài toán

Bài toán: dựa trên dữ liệu hành vi của khách hàng trên ứng dụng ngân hàng (giao dịch, hoạt động, số dư), dự đoán khách hàng có **rời bỏ dịch vụ** trong 3 tháng tiếp theo hay không.

Mô hình sử dụng: biến mục tiêu là biến nhị phân (0/1) nên **hồi quy logistic** là mô hình phù hợp.

Trọng tâm của bài: đối tượng nghiên cứu là **thuật toán tối ưu**, không phải chất lượng dự báo. Nhóm xây dựng hàm tổn thất của hồi quy logistic với các hàm phạt khác nhau:

- Hàm phạt Ridge (trọng tâm — hàm mục tiêu lõi mạnh, khả vi hai lần);
- Hàm phạt Lasso (phần mở rộng — lõi nhưng không khả vi).

Mọi so sánh dựa trên **giá trị hàm mục tiêu $f(w)$, số bước và thời gian**.

1. GIỚI THIỆU BÀI TOÁN

1.2. Mô hình hồi quy logistic

Hàm sigmoid:

$$\sigma(s) = \frac{1}{1 + e^{-s}} \in (0, 1), \quad \sigma'(s) = \sigma(s)(1 - \sigma(s)).$$

Với quan sát thứ i , đặt $s_i = w^\top x_i$ và $p_i = \sigma(s_i) = \mathbb{P}(y_i=1 \mid x_i)$.

Hàm tổn thất (ước lượng hợp lý cực đại $\Rightarrow$ log-loss trung bình, kèm hàm phạt):

$$f(w) = -\frac{1}{n} \sum_{i=1}^n \left[y_i \log p_i + (1 - y_i) \log(1 - p_i) \right] + r(w)$$

Trong đó:

- $r(w) = 0$: trường hợp không có hàm phạt;
- $r(w) = \frac{\lambda}{2} \|w\|_2^2$: hàm phạt **Ridge**;
- $r(w) = \alpha \|w\|_1$: hàm phạt **Lasso**.

1. GIỚI THIỆU BÀI TOÁN

1.3. Gradient của hàm tổn thất

Xét trường hợp không có hàm phạt. Theo quy tắc đạo hàm hợp:

$$\nabla f(w) = -\frac{1}{n} \sum_{i=1}^n \frac{\partial f}{\partial p_i} \frac{\partial p_i}{\partial w} = -\frac{1}{n} \sum_{i=1}^n \left(\frac{y_i}{p_i} - \frac{1-y_i}{1-p_i} \right) \frac{\partial p_i}{\partial w} = \frac{1}{n} \sum_{i=1}^n \frac{p_i - y_i}{p_i(1-p_i)} \frac{\partial p_i}{\partial w}$$

Mặt khác, do $\sigma'(s) = \sigma(s)(1 - \sigma(s))$:

$$\frac{\partial p_i}{\partial w} = \sigma'(s_i) \frac{\partial s_i}{\partial w} = p_i(1 - p_i) x_i \quad (1)$$

Do đó

$$\nabla f(w) = \frac{1}{n} \sum_{i=1}^n (p_i - y_i) x_i = \frac{1}{n} X^T (p - y) \quad (2)$$

Nhận xét: gradient có dạng rất gọn, chi phí tính là $O(nd)$ — một phép nhân ma trận–vector.

1. GIỚI THIỆU BÀI TOÁN

1.4. Ma trận Hessian và chứng minh tính lồi

Từ (1) và (2), đạo hàm tiếp theo w :

$$\nabla^2 f(w) = \frac{1}{n} \sum_{i=1}^n p_i(1 - p_i) x_i x_i^\top = \frac{1}{n} X^\top S X, \quad S = \text{diag}(p_i(1 - p_i)).$$

Với mọi vector $v \in \mathbb{R}^d$:

$$v^\top \nabla^2 f(w) v = \frac{1}{n} \sum_{i=1}^n \underbrace{p_i(1 - p_i)}_{>0} (x_i^\top v)^2 \geq 0.$$

- Hessian **nửa xác định dương** $\Rightarrow f$ là **hàm lồi** $\Rightarrow$ mọi điểm dừng đều là cực tiểu toàn cục: chỉ cần tìm điểm thỏa $\nabla f(w) = 0$.
- Hàm tổn thất **không có nghiệm đóng** $\Rightarrow$ bắt buộc giải bằng phương pháp lặp.

Lưu ý: gradient và Hessian của hàm phạt Ridge được cộng thêm trực tiếp: $\nabla r(w) = \lambda w$, $\nabla^2 r(w) = \lambda I$.

2. LÝ THUYẾT CÁC THUẬT TOÁN

2.1. Bốn thuật toán được thử nghiệm

Thuật toán	Công thức cập nhật	Tốc độ hội tụ	Chi phí/bước
Gradient Descent	$w^+ = w - t \nabla f(w)$	tuyến tính $(1 - \frac{1}{\kappa})^k$	$O(nd)$
Accelerated GD	thêm động lượng	tuyến tính $(1 - \frac{1}{\sqrt{\kappa}})^k$	$O(nd)$
Newton	$w^+ = w - t (\nabla^2 f)^{-1} \nabla f$	bậc hai (cục bộ)	$O(nd^2 + d^3)$
SGD / Mini-batch	gradient trên tập con	dưới tuyến tính $O(1/k)$	$O(bd)$

Nhận xét:

- Cả bốn cùng giải một hàm mục tiêu, chỉ khác **lượng thông tin** dùng ở mỗi bước: GD/AGD dùng gradient, Newton dùng thêm Hessian, SGD dùng ước lượng của gradient.
- Số bước ít hơn *không* đồng nghĩa với nhanh hơn: phải nhân với chi phí mỗi bước $\Rightarrow$ mọi so sánh trong bài đều vẽ theo **cả số bước và thời gian**.

2. LÝ THUYẾT CÁC THUẬT TOÁN

2.2. Gradient Descent và ba cách chọn độ dài bước

Chọn điểm tiếp theo theo công thức $w^+ = w - t \nabla f(w)$.

- **Độ dài bước cố định:** giá trị t không đổi qua các lần lặp. Điều kiện hội tụ: $t < \frac{2}{L}$, giá trị tốt nhất theo lý thuyết là $t = \frac{1}{L}$.

- **Backtracking (Armijo):** mỗi lần lặp bắt đầu với $t = t_{init}$, và trong khi

$$f(w - t \nabla f(w)) > f(w) - c t \|\nabla f(w)\|_2^2$$

thì cập nhật $t = \rho t$ (với $0 < \rho < 1$, $0 < c < \frac{1}{2}$); ngược lại thực hiện bước GD.

- **Acceleration (Nesterov):** chọn $w^{(0)} = w^{(-1)}$, lặp

$$v = w^{(k-1)} + \beta(w^{(k-1)} - w^{(k-2)}), \quad w^{(k)} = v - t \nabla f(v)$$

với $\beta = \frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1}$ cho hàm **lồi mạnh** (dạng $\frac{\sqrt{L} - \sqrt{\mu}}{\sqrt{L} + \sqrt{\mu}}$ trong [7], chia cho $\sqrt{\mu}$ thì ra dạng

trên), hoặc $\beta_k = \frac{k-2}{k+1}$ cho hàm **lồi thường** [1] — lược đồ thứ hai không chứa hằng số nào của bài toán, nên t ở đó là tham số tự do (mục 4.11).

2. LÝ THUYẾT CÁC THUẬT TOÁN

2.3. Vì sao Acceleration nhanh hơn: κ so với $\sqrt{\kappa}$

Với hàm lồi mạnh, số bước cần thiết để đạt sai số ϵ :

	Tốc độ hội tụ	Số bước cần
Gradient Descent	$f(w_k) - f^* \leq \left(1 - \frac{1}{\kappa}\right)^k (f(w_0) - f^*)$	$O\left(\kappa \log \frac{1}{\epsilon}\right)$
Accelerated GD	$f(w_k) - f^* \leq \left(1 - \frac{1}{\sqrt{\kappa}}\right)^k L \ w_0 - w^*\ ^2$	$O\left(\sqrt{\kappa} \log \frac{1}{\epsilon}\right)$

Nhận xét:

- Hai thuật toán có **cùng chi phí mỗi bước** $O(nd)$ (một lần tính gradient), nhưng AGD đổi κ thành $\sqrt{\kappa}$ trong tốc độ.
- Với $\kappa = 14\,069$ của bộ dữ liệu này (mục 3.3): κ so với $\sqrt{\kappa} \approx 119$ — chênh lệch **hơn hai bậc** về số bước. Mục 5.5 kiểm chứng bằng thực nghiệm: tỉ số độ dốc log–log đo được là $1,98 \approx 2$.
- Đánh đổi: quỹ đạo của AGD **không đơn điệu** (có dao động nhẹ khi gần nghiệm).

2. LÝ THUYẾT CÁC THUẬT TOÁN

2.4. Thuật toán Newton

- **Pure Newton:** $w^{(k)} = w^{(k-1)} - (\nabla^2 f(w^{(k-1)}))^{-1} \nabla f(w^{(k-1)})$.
- **Damped Newton — độ dài bước cố định:** $w^+ = w - t(\nabla^2 f(w))^{-1} \nabla f(w)$.
- **Damped Newton — Backtracking:** với hướng $v = -(\nabla^2 f(w))^{-1} \nabla f(w)$, bắt đầu $t = t_{init}$ và trong khi $f(w + tv) > f(w) + c t \nabla f(w)^\top v$ thì $t = \rho t$.

Lưu ý quan trọng khi cài đặt: không đi tìm ma trận nghịch đảo của Hessian. Thay vào đó, tìm nghiệm của hệ phương trình

$$\nabla^2 f(w) v = -\nabla f(w)$$

bằng khử Gauss (phân tích LU), Conjugate Gradient hoặc phân tích Cholesky. Đây cũng chính là ý tưởng của hai solver newton-cg và newton-cholesky trong `scikit-learn`.

Nhận xét: Newton trên hàm log-loss chính là thuật toán **IRLS** quen thuộc trong lý thuyết mô hình tuyến tính tổng quát (GLM) [8].

2. LÝ THUYẾT CÁC THUẬT TOÁN

2.5. Newton: hội tụ bậc hai cục bộ và định lý Kantorovich

Thuật toán Newton có **hai pha** rõ rệt [2]:

- **Pha damped** (khi còn ở xa nghiệm): mỗi bước làm giảm f ít nhất một lượng hằng số $\gamma > 0 \Rightarrow$ số bước của pha này bị chặn bởi $(f(w_0) - f^*)/\gamma$.
- **Pha hội tụ bậc hai** (khi đã đủ gần nghiệm): độ dài bước $t = 1$ luôn được chấp nhận và

$$\|w_{k+1} - w^*\| \leq c \|w_k - w^*\|^2$$

nghĩa là **số chữ số đúng tăng gấp đôi** sau mỗi bước.

Định lý Kantorovich (điều kiện đủ để Newton thuần hội tụ ngay từ w_0): nếu $\|\nabla^2 f(w_0)^{-1}\| \leq \beta_0$, $\|\nabla^2 f(w_0)^{-1} \nabla f(w_0)\| \leq \eta_0$, Hessian Lipschitz với hằng số M , và

$$h_0 = \beta_0 M \eta_0 \leq \frac{1}{2}$$

thì dãy lặp Newton hội tụ về w^* với tốc độ bậc hai.

Nhận xét: $h_0 \leq \frac{1}{2}$ chính là **bán kính hội tụ**: khởi tạo ngoài vùng này, Newton thuần có thể vọt lố — đó là lý do cần damping / backtracking.

2. LÝ THUYẾT CÁC THUẬT TOÁN

2.6. Stochastic Gradient Descent (SGD)

Chọn ngẫu nhiên tập con $I_k \subseteq \{1, \dots, n\}$ với $|I_k| = b \ll n$, lặp lại

$$w^{(k)} = w^{(k-1)} - t_k \cdot \frac{1}{b} \sum_{i \in I_k} \nabla f_i(w^{(k-1)}).$$

- Trung bình gradient trên mini-batch là **ước lượng không chệch** của gradient toàn bộ dữ liệu; $b = 1$ là SGD, $b > 1$ là mini-batch.
- Phương sai của ước lượng tỉ lệ σ^2/b — đây là nguồn gốc của mọi hiện tượng ở phần 4.

Hai cách chọn độ dài bước:

- **Bước cố định:** hội tụ nhanh lúc đầu nhưng dừng lại ở một **sàn phương sai**

$$\limsup_k \mathbb{E}[f(w_k) - f^*] \leq \frac{tL\sigma^2}{2\mu b} \text{ — tỉ lệ với } t/b.$$

- **Bước giảm dần** (Robbins–Monro [12]): $\sum_k t_k = \infty$, $\sum_k t_k^2 < \infty$ (ví dụ $t_k = t_0/k$ hoặc $t_0/\sqrt{k}$) $\Rightarrow$ hội tụ thật, tốc độ $O(1/k)$.

2. LÝ THUYẾT CÁC THUẬT TOÁN

2.7. Bất biến affine: vì sao phải chuẩn hóa dữ liệu

Đổi biến khả nghịch $w = Av$ và đặt $g(v) = f(Av)$. Khi đó $\nabla g(v) = A^\top \nabla f$,
 $\nabla^2 g(v) = A^\top (\nabla^2 f) A$.

Newton — bất biến affine:

$$\Delta_v = -(A^\top H A)^{-1} A^\top \nabla f = A^{-1} \Delta_w$$

Quỹ đạo Newton trên hai hệ tọa độ là *một* quỹ đạo $\Rightarrow$ số bước **không đổi** khi đổi thang đo các biến.

Nhận xét:

- Đây là lý do **bắt buộc chuẩn hóa** X : bản chất là ta *làm giúp* GD phép đổi biến mà Newton tự làm được.
- Kiểm chứng ở $\lambda = 0$ (log-loss thuần, bất biến affine *thật sự*): Newton dùng **đúng 17 bước** trên cả bộ chuẩn hóa lẫn bộ thô — hai bài toán lệch nhau 19 bậc độ lớn về thang đo. Ở $d = 415$ bản thân κ *không đo được* (Hessian tại w^* của bộ thô có 11 trị riêng không dương), nên bằng chứng nằm ở **số bước bằng nhau**, không phải ở giá trị κ .
- Với hàm phạt Ridge, số hạng $\frac{\lambda}{2} \|w\|^2$ phá vỡ bất biến affine nên Newton không còn miễn nhiễm hoàn toàn (mục 4.1).

Gradient Descent — không bất biến:

$$\Delta_v = -t A^\top \nabla f \neq A^{-1} \Delta_w$$

Bước GD phụ thuộc hoàn toàn vào đơn vị đo của từng cột $\Rightarrow$ thang đo lệch làm κ nổ và GD chậm đi rất nhiều.

2. LÝ THUYẾT CÁC THUẬT TOÁN

2.8. Newton yêu cầu ma trận đủ hạng

Thuật toán Newton phải giải hệ $\left(\frac{1}{n}X^T SX + \lambda I\right)v = -\nabla f(w)$.

- Nếu X **không đủ hạng** thì $X^T SX$ suy biến: khi $\lambda \rightarrow 0$ hệ vô định, Cholesky thất bại; khi $\lambda > 0$ hệ vẫn giải được nhưng κ rất lớn, nghiệm mất chính xác.
- GD/AGD *không báo lỗi* trong trường hợp này — chúng vẫn chạy, chỉ là nghiệm không duy nhất. Sai lầm im lặng nguy hiểm hơn sai lầm ồn ào.

Ba nguồn suy biến thường gặp và cách xử lý:

Nguồn suy biến	Cách xử lý
One-hot giữ đủ k mức + hệ số chặn	mã hóa với <code>drop_first=True</code>
Biến đa thức sinh từ biến 0/1 ($b^2 = b$)	chỉ mở rộng đa thức trên các biến liên tục
Đa cộng tuyến gần đúng (VIF cao)	cắt tỉa theo tương quan và VIF

Nhận xét: hai yêu cầu *chuẩn hóa* và *đủ hạng* quyết định toàn bộ khâu xử lý dữ liệu ở phần 3.

3. THIẾT KẾ THÍ NGHIỆM

3.1. Bộ dữ liệu

Nguồn: dữ liệu ứng dụng ngân hàng bán lẻ (ẩn danh), gồm 6 bảng nối theo mã khách hàng.

Bảng	Quy mô	Nội dung
Khách hàng	~290 nghìn dòng	thông tin nhân khẩu học (tĩnh)
Giao dịch	~1,4 triệu dòng	số lần / giá trị / loại giao dịch
Hoạt động	~16 triệu dòng	thao tác phi tài chính trên ứng dụng
Tiền gửi / Tín dụng / Thẻ	~1,3 triệu dòng	số dư theo tháng

Cách gán nhãn (tránh rò rỉ thông tin): chọn mốc T , hai cửa sổ tách rời, đều *nửa mở bên trái* để không ngày nào bị đếm hai lần:

- cửa sổ quan sát ($T - 3$ tháng, T): *chỉ* dùng để sinh đặc trưng;
- cửa sổ kết quả (T , $T + 3$ tháng]: *chỉ* dùng để sinh nhãn y .

$y_i = 1$ nếu khách hàng không phát sinh giao dịch và không có hoạt động nào trong cửa sổ kết quả.

Xếp chồng ba mốc. Với $K = H = 3$ tháng, ba cặp cửa sổ lát kín năm 2019: $T \in \{03-31, 06-30, 09-30\}$. Mỗi dòng của X là một cặp (**khách hàng, mốc**).

Kết quả: $n = 75\,026$ dòng, tỉ lệ rời bỏ 10,8% — toàn bộ dùng cho thí nghiệm tối ưu.

3. THIẾT KẾ THÍ NGHIỆM

3.2. Xây dựng ma trận thiết kế X

Bảy bước xử lý, theo đúng thứ tự:

Bước	Nội dung	Lý do
1	Gộp về 1 dòng / khách hàng	dữ liệu gốc là dạng bảng nhiều dòng theo tháng
2	Điền khuyết + $\log 1p$	không được sót NaN; giảm đuôi dày của biến tiền tệ
3	One-hot <code>drop_first</code>	bắt buộc : tránh cộng tuyến hoàn hảo (mục 2.8)
4	Chuẩn hóa z-score	bắt buộc với thuật toán bậc nhất (mục 2.7)
5	Cắt tỉa tương quan + VIF	giảm cộng tuyến <i>gần đúng</i> , hạ số điều kiện
6	QR có xoay cột	bắt buộc : hai bước trên không bắt được cộng tuyến <i>chính xác</i>
7	(tùy chọn) đặc trưng đa thức	chỉ trên biến liên tục, để không sinh cột trùng

Nhận xét: không chia train/test. Đây là bài toán tối ưu: hàm mục tiêu $f(w)$ được xác định trên *bất kỳ* cặp (X, y) nào, và mọi đại lượng ta so sánh (L , μ , κ , số bước, thời gian, $f - f^*$) đều không cần tập kiểm tra. Chia chỉ làm mất 25% dữ liệu. Bốn bước 3–6 đều xuất phát từ yêu cầu của *thuật toán*, không phải của mô hình.

3. THIẾT KẾ THÍ NGHIỆM

3.3. Các bộ dữ liệu dùng trong thí nghiệm

Bộ dữ liệu	d	Hạng	κ ($\lambda=10^{-3}$)	Dùng cho
ridge	415	415 (đủ)	14 070	thí nghiệm chính (GD/AGD/Newton/SGD)
ridge_raw	415	376	$2,66 \times 10^{19}$	đối chứng: <i>không</i> chuẩn hóa
lasso	519	506	19 700	phần mở rộng L1 (giữ biến dư thừa)

Nhận xét:

- ridge và ridge_raw có **đúng cùng các cột**, chỉ khác đơn vị đo $\Rightarrow$ so sánh hai bộ này cô lập được đúng tác động của chuẩn hóa.
- Chỉ ridge **đủ hạng** $\Rightarrow$ chỉ nó chạy được Newton. Đủ hạng không tự nhiên mà có: sau lọc tương quan và VIF vẫn sót cộng tuyến *chính xác*, phải thêm bước **QR có xoay cột** mới bảo đảm (mục 2.8).
- ridge_raw hạng 376/415 không phải vì thiếu cột, mà vì $\kappa \approx 10^{19}$ vượt dải biểu diễn của số thực 64 bit: λI thành vô hình, Hessian suy biến về mặt số học.
- lasso cố ý *không* cắt hết biến dư thừa, để hàm phạt L1 có việc để làm.

3. THIẾT KẾ THÍ NGHIỆM

3.4. Các thí nghiệm và tiêu chí so sánh

Danh sách thí nghiệm

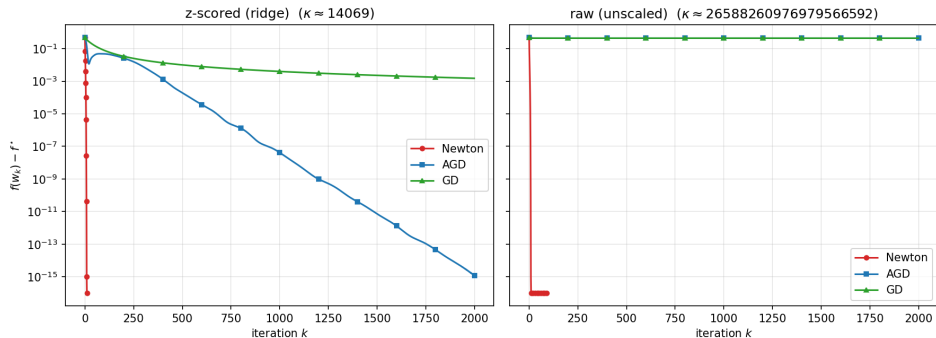
- **So sánh dữ liệu chuẩn hóa và không chuẩn hóa** — GD, AGD, Newton trên `ridge` và `ridge_raw`.
- **Thử nghiệm siêu tham số của từng thuật toán** — GD: quét độ dài bước cố định, quét (c, ρ) của backtracking; Newton: pure / damped cố định / damped backtracking; SGD: quét kích thước mini-batch và lịch giảm bước.
- **So sánh các thuật toán** với siêu tham số tốt nhất tìm được, trên cùng bộ dữ liệu chuẩn hóa.
- **Đối chiếu với `scikit-learn`** (`lbfgs`, `newton-cholesky`, `saga`).

Điều kiện giữ cố định: cùng điểm khởi tạo $w_0 = 0$; cùng $\lambda = 10^{-3}$; cùng tiêu chí dừng $\|\nabla f(w)\|_2 < \varepsilon$; cùng giá trị tham chiếu f^* (chạy Newton tới $\|\nabla f\|_2 < 10^{-14}$, mục 4.2).

Ba chỉ số báo cáo: $f(w) - f^*$ (trục tung, thang log), **số bước**, **thời gian**.

4. KẾT QUẢ: DÒ THAM SỐ BẰNG THỬ VÀ SAI

4.1. Dữ liệu chuẩn hóa và không chuẩn hóa



Nhận xét: không chuẩn hóa thì GD và AGD gần như *không giảm được gì* — sau 2 000 bước còn cách nghiệm **0,434**, trong khi bộ chuẩn hóa cho AGD $1,2 \times 10^{-15}$ và GD $1,5 \times 10^{-3}$ (κ : $1,4 \times 10^4$ so với $2,7 \times 10^{19}$, **15 bậc độ lớn**). Ở $d = 415$ Newton **không còn chạy được** chứ không phải chỉ chậm: λI chìm dưới sai số làm tròn nên Hessian suy biến về mặt số học (hạng 376/415), chậm trần 100 bước so với **11** bước trên bộ chuẩn hóa. Vì vậy chuẩn hóa là **ràng buộc bắt buộc**, không phải mẹo tinh chỉnh.

4. KẾT QUẢ: DÒ THAM SỐ BẰNG THỬ VÀ SAI

4.2. Xác định f^* — mốc tham chiếu cho mọi hình phía sau

Mọi hình trong bài vẽ $\log(f(w_k) - f^*)$, nên f^* phải được xác định **trước**, một lần, và phải *chứng nhận được*. Cách làm:

- Chạy **damped Newton** tới $\|\nabla f\| < 10^{-14}$ (backtracking mặc định $t_0 = 1$, $\rho = 0,5$, $c = 10^{-4}$; 15 vòng, 10,4s), rồi lấy giá trị tại **iterate CUỐI CÙNG**:

$$f^* = \mathbf{0,212258252065506}, \quad \|\nabla f(w^*)\| = 7,9 \times 10^{-15}.$$

- Chứng chỉ.** Với f lồi mạnh hệ số μ trên tập mức, bất đẳng thức PL cho

$$0 \leq f^* - f_{\text{true}}^* \leq \frac{\|\nabla f(w^*)\|^2}{2\mu^*} = 3,1 \times 10^{-26},$$

với $\mu^* = \lambda_{\min}(\nabla^2 f(w^*)) = 1,001 \times 10^{-3}$ *đo được*, không phải λ : hệ số chặn không bị phạt nên $\lambda I \preceq \nabla^2 f$ chỉ đúng trên các tọa độ bị phạt (mục 5.1). Sai số này nhỏ hơn sàn 10^{-16} của mọi hình **10 bậc**, nên f^* coi như chính xác tuyệt đối.

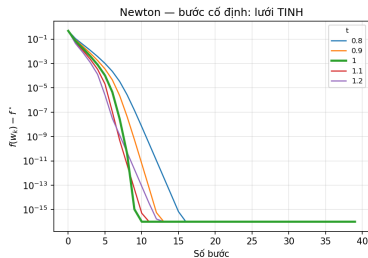
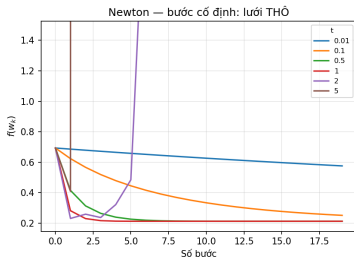
- Không lấy min của lịch sử.** $\min_k f(w_k)$ báo cáo một điểm *khác* với điểm thuật toán trả về, và nó bào mòn khoảng cách của mọi phương pháp khác theo hướng có lợi cho ta — đọc lên là thiên vị.
- Kiểm chứng độc lập:** sklearn lbfgs trên cùng hàm mục tiêu dừng ở $f^* + 2,2 \times 10^{-13}$ (mục 4.19) — hai cài đặt hoàn toàn khác nhau gặp nhau ở chữ số thứ 13.

Nhận xét: lần chạy này *chỉ* để lấy f^* — nó dùng tham số mặc định của thư viện nên **không** xuất hiện như một đường trong mục 4.17–4.18; bầy đường ở đó đều là cấu hình dò tay. `optim/reference.py`

4. KẾT QUẢ: DÒ THAM SỐ BẰNG THỦ VÀ SAI

4.3. Newton — dò bước cố định

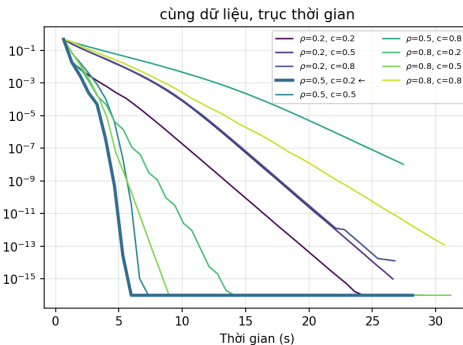
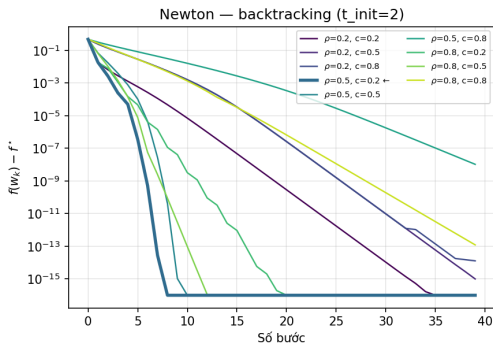
→ Chọn $t = 1$ (pure Newton)



Nhận xét: quy trình dùng chung cho *mọi* thuật toán trong mục này: một **lưới thô** [0,01; 0,1; 0,5; 1; 2; 5] chạy 20 vòng, rồi một **lưới tinh** quanh vùng trắng, rồi *nhìn hình* mà chọn — không có hàm chấm điểm nào. Lưới thô hiện đủ **ba chế độ**: $t \leq 0,1$ bò chậm ($3,6 \times 10^{-1}$; $3,9 \times 10^{-2}$), $t = 0,5-1$ về tới độ chính xác máy ($2,8 \times 10^{-17}$), còn $t = 2$ và $t = 5$ **chết thật** — dừng ở vòng 15 và vòng 5 vì $\|w\|$ vọt đi, S chạm đáy và Cholesky hỏng (xem mục 4.6). Lưới tinh thì **không phân biệt được gì**: mọi $t \in [0,8; 1,2]$ đều tới độ chính xác máy nên $f - f^*$ cuối giống hệt nhau; phải đổi tiêu chí sang **số vòng**, khi đó $t = 1$ thắng.

4. KẾT QUẢ: DÒ THAM SỐ BẰNG THỬ VÀ SAI

4.4. Newton — dò (ρ, c) của backtracking

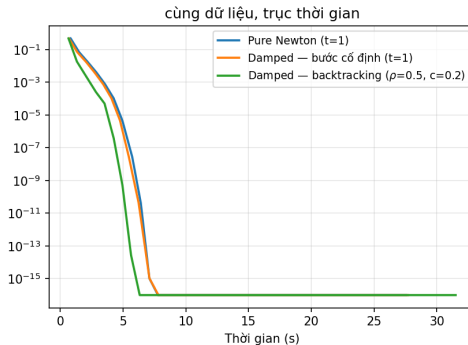
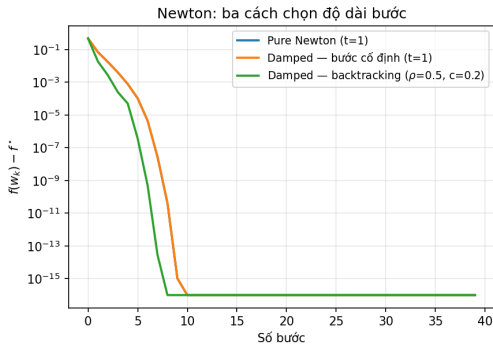


Nhận xét:

- $\rho = 0.5, c = 0.2$ thắng cả hai trực (~ 8 vòng, 5,5s) — khác hẳn GD ở mục 4.8, nơi hai trực cho hai câu trả lời khác nhau.
- c nhỏ = điều kiện Armijo dễ thỏa = giữ được bước gần 1, tức gần pure Newton — đúng thứ ta muốn khi đã ở gần nghiệm.

4. KẾT QUẢ: DÒ THAM SỐ BẰNG THỬ VÀ SAI

4.5. Newton — so sánh ba cách chọn bước

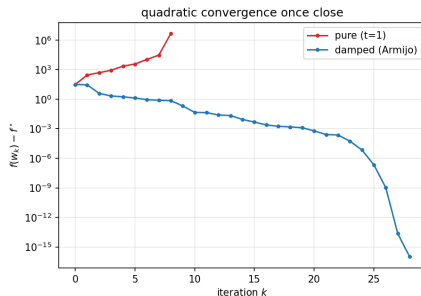
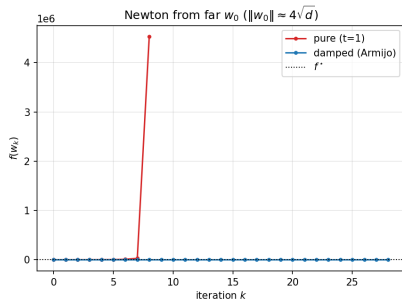


Nhận xét: (40 vòng, mỗi cách ở tham số đã dò được ở mục 4.3–4.4)

- Cả ba gần như **trùng nhau**: pure Newton đã đủ tốt khi khởi tạo gần nghiệm, damping không mua thêm được gì đáng kể.
- Trái ngược hẳn với GD ở slide trước, nơi ba cách chọn bước cách nhau cả bậc độ lớn. Lý do: hướng Newton đã mang sẵn thông tin độ cong, nên độ dài bước không còn là chỗ quyết định.
- Damping chỉ thật sự thiết yếu khi khởi tạo ở xa — xem mục 4.6.

4. KẾT QUẢ: DÒ THAM SỐ BẰNG THỬ VÀ SAI

4.6. Newton — pure và damped khi khởi tạo ở xa nghiệm

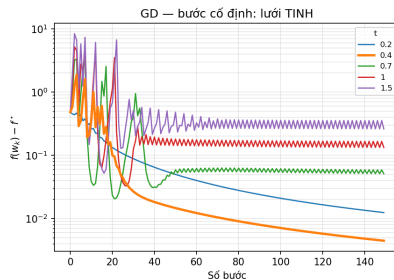
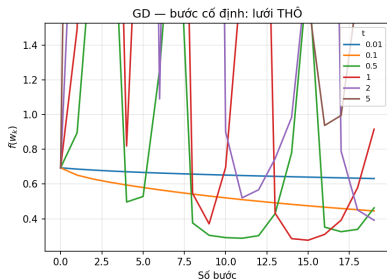


Nhận xét: pure Newton ($t = 1$) từ điểm khởi tạo xa bị **vọt lố** rồi **chết hẳn**, không phải chỉ dao động — đúng điều kiện Kantorovich $h_0 \leq \frac{1}{2}$ (mục 2.5). Cơ chế đo được: sau 8 bước $\|w\|$ leo tới $5,1 \times 10^6$, mọi sigmoid **bão hòa** nên $S = \text{diag}(p_i(1 - p_i)) \rightarrow 0$; hệ số chặn *không* có λ đở (mục 5.1) nên Hessian mất hẳn độ cong và Cholesky **thất bại thật sự** — dừng vì *không giải được hệ*, không phải vì hết ngân sách. Damped Newton từ *cùng* điểm đó cần **29 bước**. Damping vì thế **không phải tùy chọn**: khác biệt là chạy được hay không, chứ không phải nhanh hay chậm.

4. KẾT QUẢ: DÒ THAM SỐ BẰNG THỪ VÀ SAI

4.7. GD — dò độ dài bước cố định: thô rồi tinh

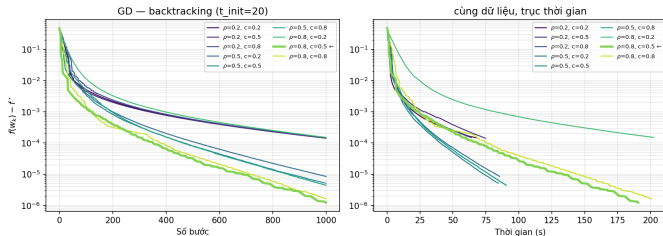
→ Chọn $t = 0,4$



Nhận xét: lưới thô ở đây **CHỈ SAI**, và đó là điều đáng nói nhất. Sau 20 vòng nó xếp $t = 2$ tốt nhất (0,179), trên cả 0,1 (0,233) và 0,5 (0,251). Nhưng lưới tinh 150 vòng lật lại: $t = 1,5$ cho 0,262, $t = 1$ cho 0,133, còn $t = 0,4$ cho $4,5 \times 10^{-3}$ — hơn $t = 1$ **30 lần**. $t = 2$ gấp 14 lần ngưỡng $2/L = 0,142$ nên nó *dao động*; ở đúng vòng thứ 20 nó tình cờ rơi vào pha thấp. Bài học: một tầng lưới không đủ, và ngân sách ngắn có thể xếp hạng ngược. Đáng chú ý: 0,4 vẫn gấp **2,8 lần** ngưỡng đơn điệu $2/L$ (mục 5.2) — đơn điệu là đủ chứ không cần.

4. KẾT QUẢ: DÒ THAM SỐ BẰNG THỬ VÀ SAI

4.8. GD — dò (ρ, c) của backtracking: hai trục, hai câu trả lời

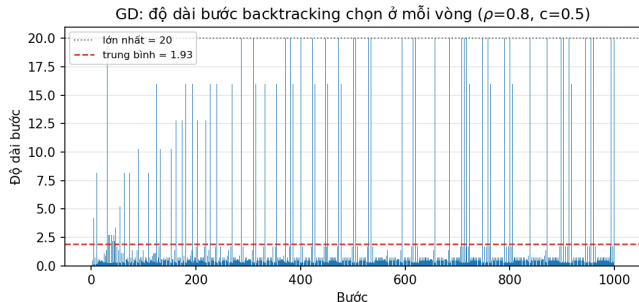


Nhận xét:

- Chín cấu hình, 1 000 vòng mỗi cấu hình. Ngân sách dài *không* xóa được mâu thuẫn — nó cho thấy đây là **biên Pareto thật**: ρ điều khiển cả hai trục cùng lúc (co chậm $\Rightarrow$ bước dài hơn $\Rightarrow$ chính xác hơn, nhưng tốn nhiều lần tính f hơn), còn c gần như không ảnh hưởng.
- **Chọn $\rho = 0,8$, $c = 0,5$ theo trục độ chính xác**: $1,2 \times 10^{-6}$, tốt nhất lược — hơn (0,5; 0,5) **4,2 lần** và hơn $\rho = 0,2$ tới **115 lần**. Giá phải trả: 190,7s so với 81,7s.
- Vì sao chọn trục này: mục 4.14 so *các thuật toán ở cùng số vòng*, nên mỗi thuật toán phải mang tham số cho *thí nghiệm tốt nhất* của nó, không phải thí nghiệm rẻ nhất. (Góc tệ nhất vẫn là $\rho = 0,8$, $c = 0,2$: vừa chậm nhất 192,3s vừa kém nhất $1,5 \times 10^{-4}$.)

4. KẾT QUẢ: DÒ THAM SỐ BẰNG THỬ VÀ SAI

4.9. GD — backtracking thật sự chọn bước nào?



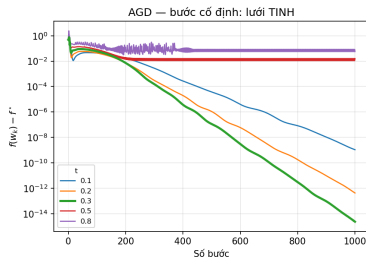
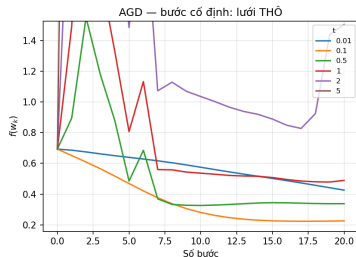
Nhận xét:

- Trung bình **1,932** — backtracking *tự tìm ra* một bước gấp **4,8 lần** bước cố định tốt nhất (0,4), và còn biết lúc nào nên bước dài.
- $t_{\text{init}} = 20$ **có lúc được chấp nhận nguyên vẹn** (bước lớn nhất = 20), trong khi $\rho = 0,5$ không bao giờ vượt quá 10. Co chậm hơn không chỉ dò *tin* hơn — nó còn *với tới* được những bước dài mà ρ lớn bỏ lỡ. Đó là nguồn gốc của $1,2 \times 10^{-6}$ so với $5,1 \times 10^{-6}$.
- Hình này trả lời câu hỏi tự nhiên nhất về backtracking — nó có thật sự đổi bước không, hay chỉ luôn lấy t_{init} — mà bảng số không trả lời được.

4. KẾT QUẢ: DÒ THAM SỐ BẰNG THỬ VÀ SAI

4.10. AGD — dò độ dài bước cố định

→ Chọn $t = 0,3$

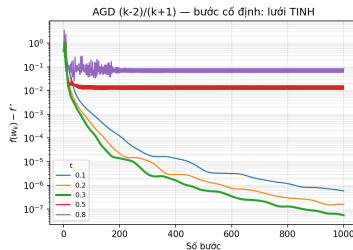
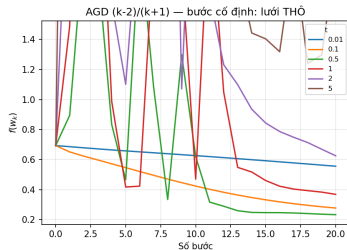


Nhận xét: $\beta = (\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1)$, lược đồ lỗi mạnh của [7]. Khác GD ở mục 4.7, lưới thô ở đây *chỉ đúng vùng* ($t = 0,1$ thắng với $1,4 \times 10^{-2}$), và lưới tinh — chạy **1 000 vòng** — tinh chỉnh lên $t = 0,3$: $2,3 \times 10^{-15}$, hơn $t = 0,2$ ($4,2 \times 10^{-13}$) **180 lần**, còn $t = 0,5$ sập xuống $1,6 \times 10^{-2}$. **Bước tốt nhất phụ thuộc ngân sách:** ở 150 vòng thì 0,2 mới thắng. Ta chọn theo 1 000 vòng vì money plot và mục 4.12–4.13 đều chạy ở đó. Lưu ý 0,3 là giá trị lớn nhất *trước vách sập* — tối ưu thật nằm trong $(0,3; 0,5)$, lưới này không tách được.

4. KẾT QUẢ: DÒ THAM SỐ BẰNG THỬ VÀ SAI

4.11. AGD — lược đồ momentum $(k-2)/(k+1)$: dò bước

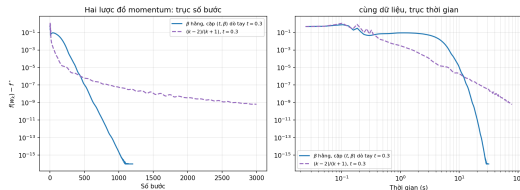
→ Chọn $t = 0,3$



Nhận xét: lược đồ Nesterov cho hàm lồi *thường* [1]: $v = w_{k-1} + \frac{k-2}{k+1}(w_{k-1} - w_{k-2})$, $w_k = v - t\nabla f(v)$, tốc độ $O(1/k^2)$. Hệ số momentum ở đây **không chứa hằng số nào của bài toán** — không L , không μ — nên t là tham số tự do thật sự, dò tay được. Lưới tinh đặt **đúng bằng** lưới của mục 4.10 để hai bảng so trực tiếp, và cùng chạy 1 000 vòng. Các giá trị tách nhau rõ: $0,1 \rightarrow 5,8 \times 10^{-7}$, $0,2 \rightarrow 1,6 \times 10^{-7}$, $t = 0,3 \rightarrow 5,6 \times 10^{-8}$, rồi 0,5 *sập* xuống $1,1 \times 10^{-2}$. Lưới thô lại chỉ sai một lần nữa — sau 20 vòng nó xếp $t = 0,5$ đầu bảng, đúng giá trị mà lưới tinh loại 5 bậc. **Trùng đúng $t = 0,3$ của mục 4.10**, nên mục 4.12 so hai lược đồ ở *cùng một bước*.

4. KẾT QUẢ: DÒ THAM SỐ BẰNG THỦ VÀ SAI

4.12. AGD — hai lược đồ momentum: ai thắng tùy ngân sách

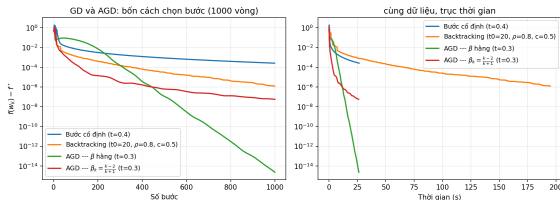


Nhận xét: hai lược đồ, mỗi cái ở bước **dỏ tay** tốt nhất của chính nó (cả hai đều $t = 0,3$ — mục 4.10 và 4.11 dò ra cùng một bước, nên chênh lệch dưới đây là do **momentum thuần túy**), trên 3 000 vòng. Chưa dùng $1/L$ ở đây — bước theo lý thuyết là chuyện của mục 5.

- $(k-2)/(k+1)$ **thắng ở giai đoạn đầu**: momentum khởi động từ 0 rồi tăng dần nên không vọt lố, trong khi β hằng đặt 0,966 ngay từ vòng 1 và có một *bước* rõ trong ~ 100 vòng đầu.
- **Nhưng nó không hội tụ**: 3 000 vòng vẫn còn $6,3 \times 10^{-10}$, vì $O(1/k^2)$ là *dưới tuyến tính* — nó không dùng tới tính lồi mạnh. β hằng vượt hẳn lên từ **vòng 445** (đo, không ước lượng bằng mắt — và từ đó không bị cắt lại) rồi rơi thẳng đứng, hội tụ sau **1 202 vòng / 31,7s**.
- Rút ra: $(k-2)/(k+1)$ dùng được khi **không ước lượng nổi** μ ; hàm thật sự lồi mạnh thì β hằng thắng đường dài. *Adaptive restart* [9] — đặt lại k mỗi khi f tăng — gỡ được đánh đổi này: nó đạt tốc độ tuyến tính mà **không cần biết** μ . Ta **chưa** cài, đây là hướng mở rộng tự nhiên.

4. KẾT QUẢ: DÒ THAM SỐ BẰNG THỦ VÀ SAI

4.13. GD và AGD — so sánh bốn cách chọn bước

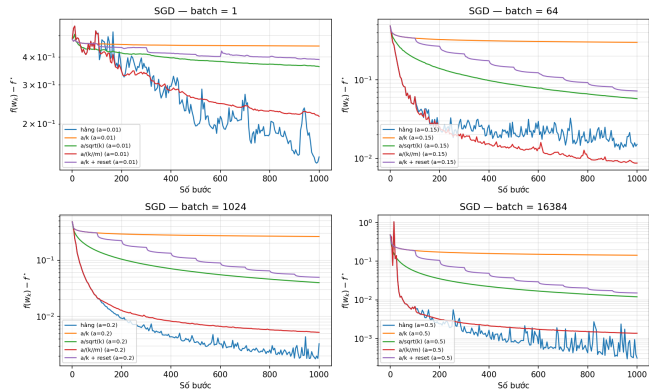


Nhận xét: (1 000 vòng, mỗi cấu hình ở tham số đã dò ở mục 4.7–4.11)

- **AGD β hằng thẳng tuyệt đối:** $2,3 \times 10^{-15}$, hơn GD bước cố định ($2,6 \times 10^{-4}$) **11 bậc** — mà *cùng thời gian* (26,3s so với 26,1s). Momentum không tốn thêm gì: cả hai đúng một gradient mỗi vòng.
- **Đảo chiều so với ngân sách ngắn.** Ở 150 vòng AGD còn *thua* GD ($2,2 \times 10^{-2}$ so với $4,5 \times 10^{-3}$). Lợi thế $\sqrt{\kappa}$ là phát biểu *tiệm cận*, và 1 000 vòng là chỗ nó hiện ra — cùng một cặp thuật toán, đổi ngân sách là đổi kết luận.
- **Backtracking** ($\rho = 0,8$, mục 4.8): tốt hơn bước cố định **210 lần** ($2,6 \times 10^{-4} \rightarrow 1,2 \times 10^{-6}$) nhưng tốn **7,4 lần thời gian** (195,0s so với 26,5s). Theo trực thời gian nó thua *cả hai* AGD rất xa — và vẫn thua AGD β hằng **9 bậc** về độ chính xác, dù đã đất gấp 7 lần.
- **Hai lược đồ momentum chạy ở cùng $t = 0,3$:** β hằng hơn $(k - 2)/(k + 1)$ ($5,6 \times 10^{-8}$) **7 bậc**. Cùng bước, cùng thời gian — khác biệt hoàn toàn do momentum, khớp mục 4.12: qua vòng 445 thì lỗi mạnh thẳng.

4. KẾT QUẢ: DÒ THAM SỐ BẰNG THỦ VÀ SAI

4.14. SGD — dò lịch bước cho từng cỡ mini-batch

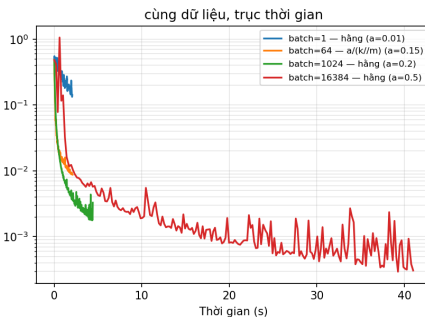
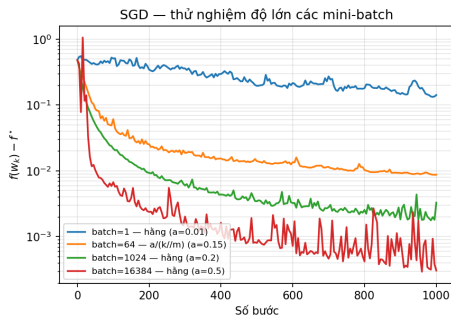


Nhận xét: (1000 bước mini-batch) α tốt nhất tăng theo batch: $0,01 \rightarrow 0,15 \rightarrow 0,2 \rightarrow 0,5$ khi

b : $1 \rightarrow 64 \rightarrow 1024 \rightarrow 16384$. α/k luôn tệ nhất (thua 1–2 bậc ở mọi cỡ batch) dù nó thỏa Robbins–Monro — thỏa điều kiện là chuyện tiệm cận, còn trong ngân sách hữu hạn thì bước đã tắt trước khi kịp đi tới đâu. Với batch lớn, **bước hằng tốt nhất**.

4. KẾT QUẢ: DÒ THAM SỐ BẰNG THỬ VÀ SAI

4.15. SGD — batch càng lớn càng tốt?



Nhận xét:

- Theo **số bước**: đúng là càng lớn càng tốt, đơn điệu — $b = 1$ dừng ở $1,4 \times 10^{-1}$, $b = 16\,384$ xuống $3,1 \times 10^{-4}$, cách nhau **3 bậc**.
- Theo **thời gian** thì *không*: $b = 1\,024$ đạt $\sim 2 \times 10^{-3}$ sau 4,5s, còn $b = 16\,384$ cần 20s mới tới cùng mức. Trong 5 giây đầu, batch vừa phải **dẫn trước** batch lớn.
- Lý do: so ở cùng số *bước* là không công bằng — một bước của $b = 16\,384$ đọc nhiều gấp 16 lần. Trực thời gian mới quy về cùng đơn vị công việc.

4. KẾT QUẢ: DÒ THAM SỐ BẰNG THỬ VÀ SAI

4.16. Tổng hợp: tham số đã dò được của từng thuật toán

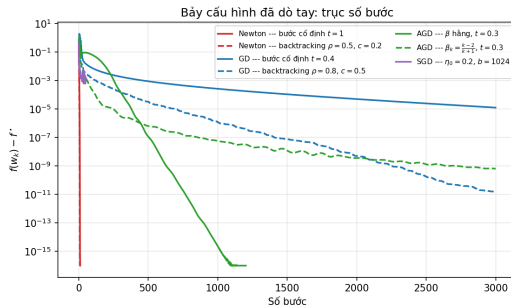
Thuật toán	Tham số chọn	Dò ở	So trong họ	Vào 4.16–4.18
Newton — bước cố định	$t = 1$ (pure)	4.2	4.4	
Newton — backtracking	$t_{init} = 2, \rho = 0,5, c = 0,2$	4.3	4.4	
GD — bước cố định	$t = 0,4$	4.6	4.13	✓
GD — backtracking	$t_{init} = 20, \rho = 0,8, c = 0,5$	4.7	4.13	
AGD — β hằng	$t = 0,3 \Rightarrow \beta = 0,966$	4.9	4.11, 4.13	✓
AGD — $\beta_k = \frac{k-2}{k+1}$	$t = 0,3$	4.10	4.11, 4.13	
SGD	$\eta_0 = 0,2, b = 1\,024$, hằng	4.13–4.14	—	✓

Nhận xét:

- **Mục 4.16–4.18 chỉ mang MỘT cấu hình cho mỗi họ** — bảy dòng ở đây là bảy *biến thể chọn bước*, còn phần so sánh sau đó trả lời câu hỏi khác: họ thuật toán nào thắng. Ba dòng không đánh dấu đã được so *trong họ* ở mục 4.5, 4.12 và 4.14.
- **Newton ở mục 4.17–4.19 là damped Newton + backtracking với $\rho = 0,5, c = 10^{-4}$, không phải cặp $c = 0,2$ ở dòng 2:** $c = 0,2$ là điều kiện Armijo chặt, gần nghiệm thì nhiều số học làm nó luôn trượt nên bước co về 0 — chạm trần 100 vòng / 95s mà không đạt $\|\nabla f\| < 10^{-14}$, so với 15 vòng / 10,4s của mặc định. Cùng lần chạy đó sinh ra f^* nên nó phải *chứng nhận được*.
- **Ba lần trực đo quyết định câu trả lời:** Newton so bằng số vòng; GD-backtracking chọn giữa *độ chính xác* và *thời gian* (biên Pareto); SGD đọc theo *thời gian*. Và không tham số nào trùng cận lý thuyết ($0,4$ so với $2/L = 0,142$; $0,3$ so với $1/L = 0,071$) — mục 5 chứng minh vì sao.

4. KẾT QUẢ: DÒ THAM SỐ BẰNG THỦ VÀ SAI

4.17. So sánh cả bảy cấu hình — theo số bước

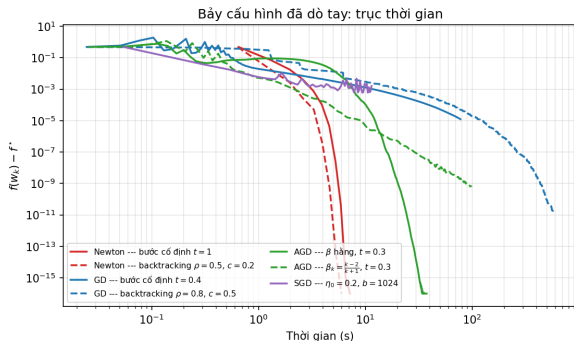


Nhận xét: bảy cấu hình của bảng 4.16, *không* cấu hình nào lấy từ nơi khác. Cùng tiêu chí dừng $\|\nabla f\| < 10^{-10}$, cùng trần 3 000 vòng, cùng f^* của mục 4.2. Màu = họ thuật toán, nét = cách chọn bước.

- **Hai đường Newton dựng đứng ở mép trái:** 10 và 8 vòng. Backtracking dò tay *nhANH hơn* pure Newton — $c = 0,2$ cho phép nhận bước dài hơn 1 ở pha đầu.
- **Chỉ 3/7 đạt tiêu chí dừng:** hai Newton và AGD β hằng (1 202 vòng). Bốn cấu hình còn lại chạm trần.
- **Trong cùng một họ, cách chọn bước đổi kết quả tới 6 bậc:** GD $1,2 \times 10^{-5}$ so với $1,5 \times 10^{-11}$; AGD 0 so với $6,3 \times 10^{-10}$. Chênh lệch *trong* họ lớn hơn chênh lệch *giữa* vài họ.

4. KẾT QUẢ: DÒ THAM SỐ BẰNG THỬ VÀ SAI

4.18. So sánh cả bảy cấu hình — theo thời gian



Cấu hình	Bước	$f - f^*$	Giây
Newton bt (0,5; 0,2)	8	0	6,0
Newton $t = 1$	10	0	7,2
SGD	50 ep	$1,6 \times 10^{-3}$	11,4
AGD β hằng	1 202	0	37,5
GD $t = 0,4$	>3 000	$1,2 \times 10^{-5}$	77,7
AGD $(k-2)/(k+1)$	>3 000	$6,3 \times 10^{-10}$	99,1
GD backtracking	>3 000	$1,5 \times 10^{-11}$	583,2

Nhận xét:

- **Newton thắng cả hai trực** dù mỗi bước của nó đắt nhất: 0,75s một bước, gấp **24 lần** một bước AGD — nhưng cần ít hơn **150 lần** số bước.
- **Đổi trực là đổi hạng.** GD-backtracking chính xác thứ ba ($1,5 \times 10^{-11}$) nhưng *chậm nhất bảng* (583,2s, gấp 7,5 lần GD bước cố định); ngược lại SGD thứ ba về thời gian nhưng bét về độ chính xác.
- Toàn bộ luận điểm của bài trong một hình: **chi phí mỗi vòng nhân số vòng** mới ra thời gian, và hai thừa số đó đi ngược chiều nhau.

4. KẾT QUẢ: DÒ THAM SỐ BẰNG THỬ VÀ SAI

4.19. Đối chiếu với thư viện `scikit-learn`

Khớp đúng dạng hàm mục tiêu: `scikit-learn` tối thiểu $\frac{1}{2}\|w\|^2 + C\sum_i \ell_i$, hàm của nhóm lấy *trung bình*

$\Rightarrow C = 1/(n\lambda)$; hệ số chặn không bị phạt ở cả hai phía.

Solver	$f - f^*$	Số bước	Thời gian (s)
<code>sklearn lbfgs</code>	$2,2 \times 10^{-13}$	225	3,31
<code>sklearn newton-cholesky</code>	$2,8 \times 10^{-17}$	10	6,84
Newton (nhóm tự cài)	0	11	10,21
SGD (nhóm tự cài) [†]	$1,6 \times 10^{-3}$	51 epoch	10,60
Accelerated GD (nhóm tự cài)	$5,6 \times 10^{-17}$	1 335	39,7
GD (nhóm tự cài) [†]	$2,1 \times 10^{-13}$	20 000	540,9
<code>sklearn saga</code>	$1,2 \times 10^{-11}$	5 000	852

[†] chạm trần ngân sách, chưa đạt $\|\nabla f\| < 10^{-11}$. Mọi dòng "nhóm tự cài" chạy ở tham số đã dò ở mục 4.16.

Nhận xét:

- Newton của nhóm (11 bước) khớp `newton-cholesky` (10 bước) tới độ chính xác máy: chênh lệch 10,21s so với 6,84s chỉ đến từ cài đặt (BLAS/Cython), *không* từ thuật toán [10].
- **Đảo chiều so với bài toán nhỏ:** ở $d = 39$ Newton nhanh nhất, ở $d = 415$ **lbfgs thắng** (3,31s) — quasi-Newton tốn $O(nd)$ mỗi vòng và không bao giờ dựng Hessian, nên 225 vòng rẻ hơn 10 vòng đắt.
- **GD tới được cùng độ chính xác — nhưng mất 541s, gấp 163 lần** lbfgs cho cùng $\sim 10^{-13}$: cái giá của tốc độ tuyến tính khi $\kappa = 14\,069$. Còn **SGD dừng ở** $1,6 \times 10^{-3}$, kém 10 bậc, ở đúng mốc thời gian của Newton — rẻ để tới *gần*, không phải tới nghiệm.

5. HẲNG SỐ LIPSCHITZ VÀ CHỌN BƯỚC

Mục 4 chọn bước bằng cách *dò tay*. Mục này hỏi: lý thuyết nói gì, và nó cách thực nghiệm bao xa?

5. HÀNG SỐ LIPSCHITZ VÀ CHỌN BƯỚC THEO LÝ THUYẾT

5.1. Hàm phạt Ridge và ba hằng số L, μ, κ

Với hàm phạt Ridge, $\nabla^2 f(w) = \frac{1}{n} X^\top S X + \lambda I$. Do $0 < p(1-p) \leq \frac{1}{4}$:

$$\lambda I \preceq \nabla^2 f(w) \preceq \underbrace{\left(\frac{1}{4n} \lambda_{\max}(X^\top X) + \lambda \right)}_L I$$

- $\mu \geq \lambda > 0$: hàm **lồi mạnh** $\Rightarrow$ nghiệm w^* **tồn tại và duy nhất**.
- L : hằng số **trơn** (Lipschitz của gradient) $\Rightarrow$ quyết định độ dài bước tối đa.
- $\kappa = L/\mu$: **số điều kiện** — xuất hiện trong tốc độ hội tụ của mọi thuật toán bậc nhất.

Chính xác hơn: hệ số chặn **không bị phạt**, nên số hạng Ridge thực ra là $\lambda \text{diag}(r)$ với một số 0 ở vị trí hệ số chặn. Bất đẳng thức bên trái chỉ đúng trên các tọa độ bị phạt; theo hướng hệ số chặn, độ cong là $\frac{1}{n} \sum_i p_i(1-p_i) \rightarrow 0$ khi $\|w\| \rightarrow \infty$. Vậy f lồi mạnh trên mọi **tập mức bị chặn**, không phải toàn cục — tại nghiệm đo được $\mu_* = 1,00 \times 10^{-3}$, tức gần **đúng bằng** λ : ở $d = 415$ số hạng Ridge là thứ duy nhất chống đỡ hướng cong yếu nhất. Điều này *không* vô hại: xem mục 4.6; chứng minh cận L ở mục 5.2.

5. HÀNG SỐ LIPSCHITZ VÀ CHỌN BƯỚC THEO LÝ THUYẾT

5.2. Chứng minh cận L , và vì sao ngưỡng lại là $2/L$

Bước 1 — chặn độ cong theo từng mẫu. Với $p = \sigma(z)$ thì $p(1-p)$ đạt cực đại tại $p = \frac{1}{2}$, nên $0 < p_i(1-p_i) \leq \frac{1}{4}$ với mọi i và mọi w .

Bước 2 — từ đó chặn dạng toàn phương. Với mọi $v \neq 0$:

$$v^T \left(\frac{1}{n} X^T S X \right) v = \frac{1}{n} \sum_i p_i(1-p_i) (x_i^T v)^2 \leq \frac{1}{4n} \sum_i (x_i^T v)^2 = \frac{1}{4} v^T \left(\frac{1}{n} X^T X \right) v \leq \frac{\lambda_{\max}(\frac{1}{n} X^T X)}{4} \|v\|^2.$$

Cộng số hạng Ridge $\lambda \text{diag}(r) \preceq \lambda I$ được $\nabla^2 f(w) \preceq L I$ với $L = \frac{1}{4} \lambda_{\max}(\frac{1}{n} X^T X) + \lambda$, **đúng với mọi w .**

Bước 3 — $\nabla^2 f \preceq L I$ chính là tính L -trơn. Khai triển Taylor bậc hai với phần dư Lagrange cho *bổ đề giảm*: $f(y) \leq f(x) + \nabla f(x)^T (y-x) + \frac{L}{2} \|y-x\|^2$.

Bước 4 — thay $y = x - t \nabla f(x)$ vào bổ đề giảm:

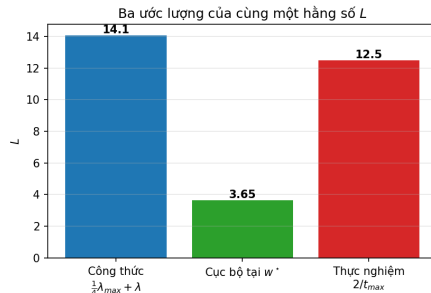
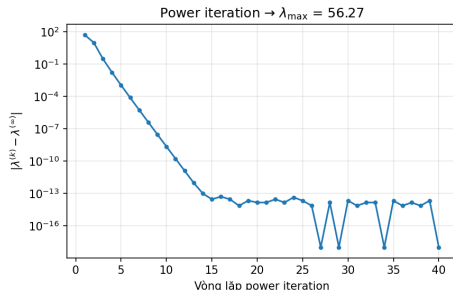
$$f(x - t \nabla f(x)) \leq f(x) - t \left(1 - \frac{tL}{2} \right) \|\nabla f(x)\|^2.$$

Hệ số $t(1 - tL/2)$ dương $\iff t < 2/L$. **Đó là toàn bộ nguồn gốc của ngưỡng $2/L$:** nó bảo đảm f giảm ở mỗi bước, chứ không nói gì về việc bước nào *tốt nhất*.

Nhận xét: cả bốn bước đều lấy bất đẳng thức theo hướng *an toàn* (bước 1 lấy $\frac{1}{4}$ cho mọi mẫu dù $p_i(1-p_i)$ nhỏ hơn nhiều khi rời w_0 ; bước 2 lấy $\lambda_{\max}$ cho mọi hướng), nên $2/L$ là điều kiện **đủ**, không **cần**. Đo được: GD dò tay ra 0,4, gấp **2,8** lần $2/L$ (mục 4.7); AGD ở cặp dò tay $t = 0,3$ cần **1 202 vòng / 31,7s** so với 2 400 vòng / 63,1s ở bước $1/L$ — nhanh hơn **2,0 lần** trên cả hai trục. Cái *phá* AGD là đổi t mà **giữ nguyên** β , không phải rời khỏi $1/L$.

5. HẰNG SỐ LIPSCHITZ VÀ CHỌN BƯỚC THEO LÝ THUYẾT

5.3. Xác định hằng số trơn L — bốn đường, một con số

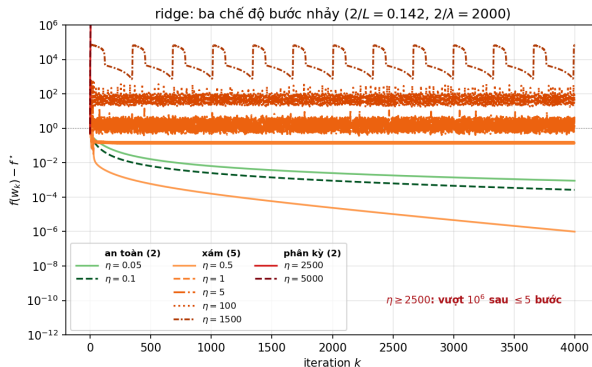


Nhận xét:

- **Công thức** (dùng $0 < p(1-p) \leq \frac{1}{4}$): $L = \frac{1}{4}\lambda_{\max}(\frac{1}{n}X^T X) + \lambda = 14,07$. Đúng với *mọi* w — an toàn nhưng bi quan.
- **Power iteration** cho cùng $\lambda_{\max} = 56,27$ nhưng chỉ tốn $O(nd)$ mỗi vòng thay vì $O(nd^2 + d^3)$ của eigvalsh, và hội tụ tới độ chính xác máy sau **15 vòng**. Đây là cách duy nhất còn rẻ khi d lớn hơn nữa.
- **Độ cong cục bộ tại w^*** : $L_* = 3,65$ — nhỏ hơn **3,9 lần**, vì tại nghiệm thì $p(1-p)$ đã rời xa $\frac{1}{4}$.
- **Thực nghiệm**: bước cố định lớn nhất mà GD còn *giảm đơn điệu* nằm trong khoảng $(0,16; 0,20)$, trong khi lý thuyết nói ngưỡng là $2/L = 0,142$. Cận lý thuyết chỉ bi quan 1,1–1,4 lần — **khá chặt**.

5. HÀNG SỐ LIPSCHITZ VÀ CHỌN BƯỚC THEO LÝ THUYẾT

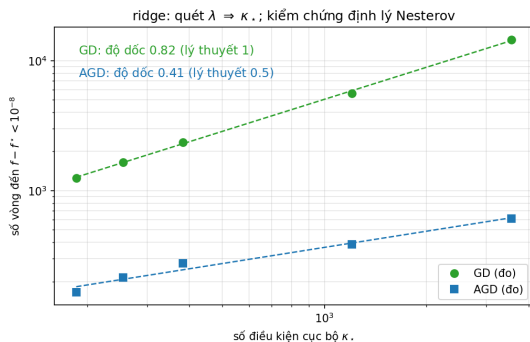
5.4. Ba vùng độ dài bước quanh $2/L$ và $2/\lambda$



Nhận xét: ba vùng, mỗi vùng một cột chú giải. $t < 2/L = 0,142$ (xanh): giảm **đơn điệu** đúng lý thuyết — nhưng *chậm nhất* hình. $2/L < t < 2/\lambda$ (cam, 5 đường): **dao động** nhưng hữu hạn, và $\eta = 0,5$ về tới 10^{-6} — nhanh hơn cả hai đường "an toàn"; bước dò tay $t = 0,4$ của mục 4.7 nằm ở đây. $t > 2/\lambda = 2000$ (đỏ): **phân kỳ** thật sự, vượt 10^6 sau ≤ 5 bước nên chạy khỏi khung ngay ở mép trái.

5. HÀNG SỐ LIPSCHITZ VÀ CHỌN BƯỚC THEO LÝ THUYẾT

5.5. Kiểm chứng tốc độ lý thuyết: GD tỉ lệ κ , AGD tỉ lệ $\sqrt{\kappa}$



Thay đổi λ để κ_* trải $\sim 1,3$ bậc, đếm số bước tới $f - f^* < 10^{-8}$:

κ_*	GD	AGD
185,5	1 252	165
255,0	1 658	214
382,6	2 362	275
1 204,5	5 613	386
3 559,9	14 541	609

Nhận xét:

- Độ dốc log-log: GD 0,819, AGD 0,413 — cả hai thấp hơn lý thuyết (1 và $\frac{1}{2}$) vì đó là **cận trên**.
- Nhưng **tỉ số** là $0,819/0,413 = 1,98$, gần đúng bằng 2 — đây mới là điều κ vs $\sqrt{\kappa}$ tiên đoán.
- κ_* tăng 19 lần: GD tốn thêm **11,6** lần bước, AGD **3,7**.

Đo trên mẫu 15 000/75 026 dòng — κ là tính chất của hình học đặc trưng, không phải của cỡ mẫu (lệch 2,4%), mà GD cần $O(\kappa \log \frac{1}{\epsilon})$ vòng.

6. MỘT SỐ THUẬT TOÁN TỐI ƯU KHÁC

6.1. Họ các thuật toán Ada

AdaGrad (*Adaptive Gradient*) [3]:

$$G_k = G_{k-1} + g_k^2, \quad w_{k+1} = w_k - \frac{\eta}{\sqrt{G_k} + \epsilon} g_k$$

Trong đó g_k là gradient tại bước k , G_k lưu tổng bình phương các gradient trước đó, ϵ là hằng số nhỏ tránh chia cho 0. Mọi phép toán thực hiện *theo từng tọa độ*.

AdaDelta: tương tự RMSprop nhưng bổ sung Δw_k nên **không cần** tham số độ dài bước:

$$\Delta w_k = - \frac{\sqrt{E[\Delta w^2]_{k-1} + \epsilon}}{\sqrt{E[g^2]_k + \epsilon}} g_k$$

RMSprop: hạn chế của AdaGrad là G_k chỉ tăng, làm độ dài bước tắt dần. RMSprop thay G_k bằng trung bình trọng số của bình phương gradient:

$$E[g^2]_k = \gamma E[g^2]_{k-1} + (1 - \gamma) g_k^2$$

Nhận xét:

- Bản chất là **đổi thang đo theo từng tọa độ** — họ hàng rẻ tiền của Newton: $O(d)$ mỗi bước thay vì $O(nd^2 + d^3)$.
- Chuẩn hóa z-score (mục 3.2) đã cân bằng *thang đo* các cột, nhưng *không* cân bằng độ cong cục bộ $p_i(1 - p_i)$ — phần việc còn lại này chính là chỗ bước thích nghi vẫn giúp được (mục 6.6).
- Cận lý thuyết của AdaGrad là $O(1/k)$, yếu hơn tốc độ tuyến tính của GD — nhưng đó là *cận trên* cho trường hợp xấu nhất.

6. MỘT SỐ THUẬT TOÁN TỐI ƯU KHÁC

6.2. Họ các thuật toán Adam

Adam (*Adaptive Moment Estimation*) [5] = RMSprop + Momentum:

$$m_k = \beta_1 m_{k-1} + (1 - \beta_1) g_k$$

$$v_k = \beta_2 v_{k-1} + (1 - \beta_2) g_k^2$$

$$\hat{m}_k = \frac{m_k}{1 - \beta_1^k}, \quad \hat{v}_k = \frac{v_k}{1 - \beta_2^k}$$

$$w_{k+1} = w_k - \frac{\eta}{\sqrt{\hat{v}_k} + \epsilon} \hat{m}_k$$

β_1, β_2 là tỷ lệ trọng số của các ước lượng moment (thường là 0,9 và 0,999).

AdamW (*Adam with Weight Decay*) [6]: tách hàm phạt ra khỏi gradient

$$w_{k+1} = w_k - \eta \left(\frac{\hat{m}_k}{\sqrt{\hat{v}_k} + \epsilon} + \lambda w_k \right)$$

Nadam: thay động lượng thường bằng động lượng Nesterov

$$w_{k+1} = w_k - \eta \frac{\beta_1 \hat{m}_k + (1 - \beta_1) g_k}{\sqrt{\hat{v}_k} + \epsilon}$$

AMSGrad [11]: dùng $\hat{v}_k = \max(\hat{v}_{k-1}, v_k)$ để mẫu số không giảm — vá lại chứng minh hội tụ của Adam gốc.

6. MỘT SỐ THUẬT TOÁN TỐI ƯU KHÁC

6.3. Hai biến thể của Adam — cơ chế

AMSGrad — kẹp $\hat{v}_k = \max(\hat{v}_{k-1}, v_k)$.

- Adam lấy $\hat{v}_k$ là trung bình trượt, nên mẫu số $\sqrt{\hat{v}_k}$ *có thể giảm* $\Rightarrow$ bước hiệu dụng $\eta / \sqrt{\hat{v}_k}$ có thể *tăng* trở lại giữa chừng. Đó chính là chỗ chứng minh hội tụ của Adam bị thủng.
- Phép kẹp lấy max làm $\sqrt{\hat{v}_k}$ **không giảm**, nên bước hiệu dụng đơn điệu không tăng — điều kiện kiểu Robbins–Monro được khôi phục. RMSprop *không* có tính chất này.

AdamW — weight decay tách rời, và hệ quả không hiển nhiên của nó.

- Thay vì cộng λw vào gradient, AdamW trừ thẳng $\eta \lambda w$ khỏi w . Điểm dừng vì thế thỏa $\nabla g(w) + \lambda(\sqrt{\hat{v}} + \epsilon) w = 0$, trong khi $\arg \min f$ thỏa $\nabla g(w) + \lambda w = 0$.
- Hai phương trình này chỉ trùng nhau khi $\sqrt{\hat{v}} + \epsilon = 1$. Nói cách khác AdamW **không tối thiểu hóa cùng một hàm mục tiêu** với các thuật toán còn lại — nó không thuộc cùng một cuộc đua.

Nhận xét: số đo kiểm chứng cả hai điều trên ở mục 6.8.

6. MỘT SỐ THUẬT TOÁN TỐI ƯU KHÁC

6.4. Dò η cho cả họ — cùng một quy trình thô rồi tinh

Cả năm thuật toán dò theo đúng quy trình của mục 4.3: lưới thô cách nhau một bậc 10, rồi lưới tinh bao quanh giá trị thắng, rồi nhìn hình mà chọn.

	η chọn	$f - f^*$ (150 vòng)	Ghi chú
AMSGrad	0,2	$1,2 \cdot 10^{-3}$	cực tiểu nội
Adam	0,12	$4,5 \cdot 10^{-4}$	phải nói lưới lên mới ra
AdaGrad	0,3	$7,0 \cdot 10^{-3}$	cực tiểu nội ngay vòng đầu
AdamW	0,08	$5,6 \cdot 10^{-3}$	phải nói lưới lên mới ra
RMSprop	0,02	$5,1 \cdot 10^{-2}$	phải nói lưới lên; vẫn tệ nhất họ

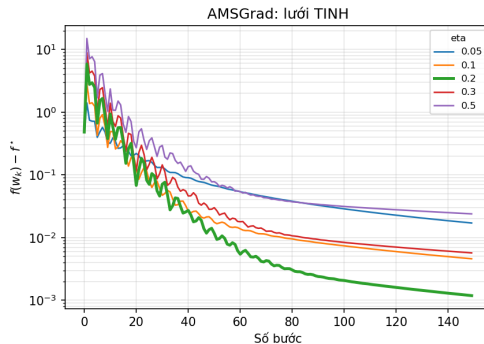
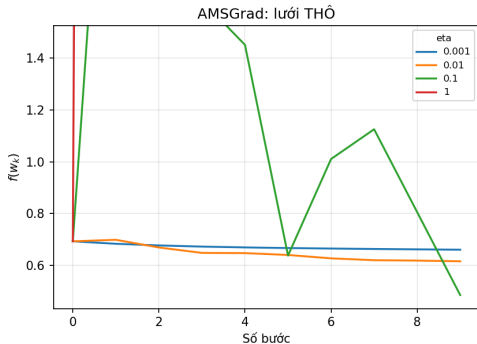
Nhận xét:

- Vòng dò đầu đặt lưới tinh theo kết quả cũ ở 3 000 vòng, và **ba trong năm thuật toán chạm mép trên**: ở ngân sách 150 vòng thì bước tối ưu *lớn hơn hẳn* (Adam $0,01 \rightarrow 0,12$, gấp 12 lần).
- Nên không thể chép tham số từ một lần chạy khác — η tốt nhất phụ thuộc **cả ngân sách**, không riêng hàm mục tiêu. Cả năm giá trị cuối đều là cực tiểu *nội*.

6. MỘT SỐ THUẬT TOÁN TỐI ƯU KHÁC

6.5. Dò η — minh họa với AMSGrad

→ Chọn $\eta = 0,2$



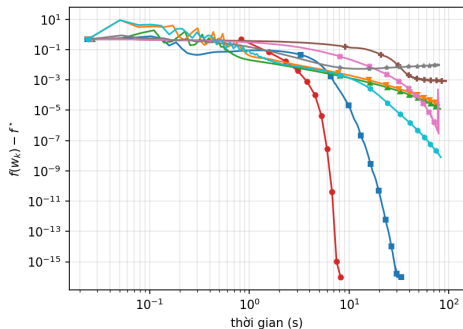
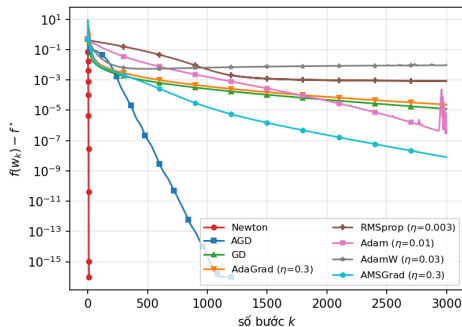
Nhận xét:

- Lưới thô (trục thẳng): $\eta = 1$ vọt lên, $\eta \leq 0,01$ gần như đứng yên — vùng cần dò nằm quanh 0,1.
- Lưới tinh (thang log): $\eta = 0,2$ rõ ràng là cực tiểu *nội* — 0,1 cho $4,6 \cdot 10^{-3}$ và 0,3 cho $5,7 \cdot 10^{-3}$, đều tệ hơn.
- Bốn thuật toán còn lại có hình tương tự trong `artifacts/figures/tune_ada_*.png`.

6. MỘT SỐ THUẬT TOÁN TỐI ƯU KHÁC

6.6. Kết quả: họ Ada/Adam so với GD, AGD, Newton

ridge: họ Ada/Adam so với GD, AGD, Newton ($\lambda=0.001$, mỗi thuật toán ở bước tốt nhất)



Nhận xét: tám đường trên cùng một hàm mục tiêu, cùng điểm xuất phát, cùng trần 3 000 bước; mỗi thuật toán thích nghi chạy ở η tốt nhất của riêng nó và GD chạy ở bước dò tay $t = 0,4$. **Chỉ AGD và Newton chạm tiêu chí dừng;** cả họ bước-thích-nghi đều nằm lại trên trần, còn AGD về đúng f^* sau 1 202 vòng. Số liệu và thứ hạng ở mục 6.7.

6. MỘT SỐ THUẬT TOÁN TỐI ƯU KHÁC

6.7. Họ Ada/Adam — bảng số và thứ hạng

	η	Bước	$f - f^*$
Newton	—	10	0
AGD	—	1 202	0
AMSGrad	0,3	×	$8,0 \cdot 10^{-9}$
Adam	0,01	×	$5,7 \cdot 10^{-7}$
GD	—	×	$1,2 \cdot 10^{-5}$
AdaGrad	0,3	×	$2,3 \cdot 10^{-5}$
RMSprop	0,003	×	$8,7 \cdot 10^{-4}$
AdamW [‡]	0,03	×	$1,0 \cdot 10^{-2}$

Nhận xét: $\times$ = chạm trần 3 000 bước, chưa đạt tiêu chí dừng. GD và AGD chạy bước **dò tay** (0,4 và 0,3) đúng như money plot. Cột η ở đây khác mục 6.4–6.5 (AMSGrad 0,3 so với 0,2, Adam 0,01 so với 0,12) vì nó được quét lại ở **ngân sách 3 000 vòng**, còn mục 6.4 dò ở 150 vòng — chính hiện tượng mà 6.4 đã cảnh báo: η tốt nhất phụ thuộc ngân sách.

- **Chỉ AGD và Newton hội tụ.** Không thuật toán bước-thích-nghi nào về đích, dù AMSGrad (tốt nhất trong họ) hơn GD hơn **3 bậc**.
- **Ba trong năm thuật toán thích nghi THUA GD thuần:** AdaGrad ($2,3 \cdot 10^{-5}$), RMSprop, AdamW đều tệ hơn $1,2 \cdot 10^{-5}$ của GD ở $t = 0,4$. Chỉ Adam và AMSGrad vượt được. Cơ chế thích nghi *không* miễn phí: nó đổi một bước dở lấy một bước trung bình, chứ không thay được việc dò bước tử tế.
- **ĐẢO CHIỀU so với bài toán nhỏ.** Ở $d = 39$ Adam về đích sau 413 bước còn AGD cần 800; ở $d = 415$, cùng đoạn code, AGD hội tụ sau 2 400 bước còn Adam thì không. *Đừng kết luận từ bài toán nhỏ:* khi κ tăng, điều chỉnh theo đường chéo không thay được thông tin cong đầy đủ.
- [‡]AdamW: xem mục 6.8.

6. MỘT SỐ THUẬT TOÁN TỐI ƯU KHÁC

6.8. Hai biến thể của Adam — số đo nói gì

AMSGrad — cơ chế ở mục 6.3 hoạt động đúng như hứa.

- $8,0 \times 10^{-9}$ so với $5,7 \times 10^{-7}$ của Adam — **tốt hơn 72 lần**, và là kết quả tốt nhất trong cả họ bước-thích-nghi.
- Mẫu số $\sqrt{\hat{v}_k}$ không giảm nên bước hiệu dụng đơn điệu không tăng. Đó đúng là tính chất RMSprop thiếu, và cũng là lý do RMSprop dừng ở sàn $8,7 \times 10^{-4}$. Công thức ở mục 6.3 giờ có số đo đi kèm.

AdamW — $1,0 \times 10^{-2}$ **KHÔNG** có nghĩa là tối ưu kém.

- Nó về đúng nghiệm của *một bài toán khác* (mục 6.3). Hai bằng chứng đo được: sàn **không đổi** theo η ($1,0$ – $1,1 \times 10^{-2}$ với mọi $\eta \in [0,003; 0,1]$) — nếu là chuyện tối ưu thì η phải ảnh hưởng; và khoảng cách còn **tăng** theo thời gian ($6,5 \times 10^{-3}$ ở bước 1 000 $\rightarrow 1,0 \times 10^{-2}$ ở bước 3 000) — một thuật toán tối ưu kém thì đứng yên, không đi xa dần.
- Bài học: khi so sánh thuật toán, phải kiểm tra chúng có cùng *điểm dừng* không — nếu không thì bảng xếp hạng đang so hai bài toán khác nhau.

7. MỞ RỘNG SANG BÀI TOÁN LASSO

7.1. Bài toán và phương pháp Subgradient

Với hàm phạt Lasso, hàm mục tiêu trở thành

$$F(w) = \underbrace{\frac{1}{n} \sum_{i=1}^n \ell_i(w)}_{\text{trơn, lồi}} + \underbrace{\alpha \|w\|_1}_{\text{lồi, không trơn}}$$

Hàm vẫn lồi nhưng **không khả vi** tại mọi điểm có tọa độ bằng 0 $\Rightarrow$ không dùng được GD và Newton như cũ. Điều kiện tối ưu đổi từ $\nabla F = 0$ thành $0 \in \partial F(w^*)$.

Phương pháp Subgradient: giống Gradient Descent nhưng thay gradient bằng subgradient

$$w^{(k)} = w^{(k-1)} - t_k g^{(k-1)}, \quad g^{(k-1)} \in \partial F(w^{(k-1)})$$

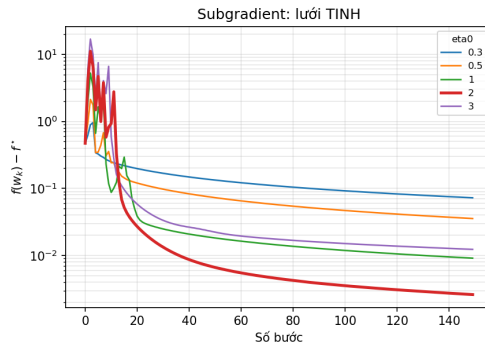
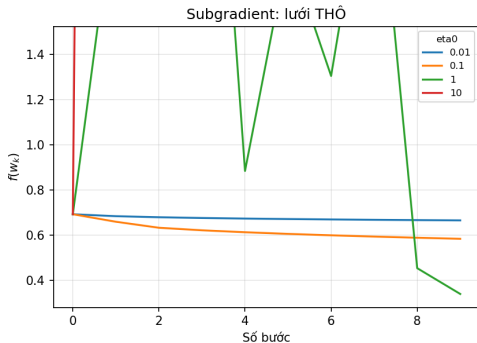
với $\partial \|w\|_1$ lấy $\text{sign}(w_j)$ nếu $w_j \neq 0$ và một giá trị bất kỳ trong $[-1, 1]$ nếu $w_j = 0$.

Nhận xét: tốc độ chỉ $O(1/\sqrt{k})$, quỹ đạo không đơn điệu, và nghiệm **không thừa thật sự** (các hệ số chỉ tiến *gần* 0 chứ không *bằng* 0).

7. MỞ RỘNG SANG BÀI TOÁN LASSO

7.2. Dò η_0 cho subgradient

→ Chọn $\eta_0 = 2$



Nhận xét:

- $\eta_0 = 2$, cực tiểu nội: 1,0 cho $9,1 \cdot 10^{-3}$, 3,0 cho $1,2 \cdot 10^{-2}$, còn 2,0 cho $2,6 \cdot 10^{-3}$.
- Dò η_0 cẩn thận là bắt buộc ở đây: một phương pháp subgradient trông tệ vì bị cho bước sai thì không chứng minh được điều gì về subgradient cả.

7. MỞ RỘNG SANG BÀI TOÁN LASSO

7.3. Phương pháp Proximal Gradient Descent

Tách $F = f_0 + h$ với f_0 trơn và $h = \alpha \|w\|_1$. Mỗi bước làm một bước gradient trên f_0 rồi chiếu qua toán tử prox của h :

$$w^{(k)} = \text{prox}_{t_k h} \left(w^{(k-1)} - t_k \nabla f_0(w^{(k-1)}) \right), \quad \text{prox}_{t\alpha \|\cdot\|_1}(v)_j = \text{sign}(v_j) \max(|v_j| - t\alpha, 0)$$

Toán tử này gọi là **soft-thresholding**: nó đặt các tọa độ *đúng bằng 0* $\Rightarrow$ nghiệm thưa thật sự.

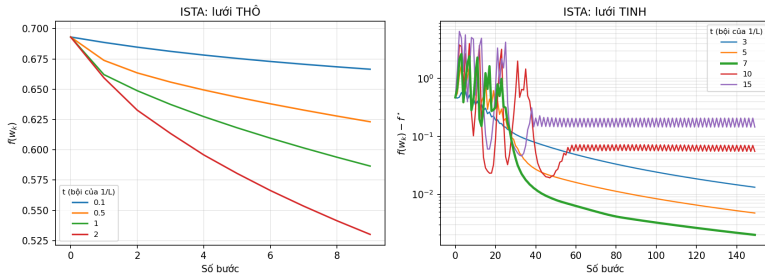
- **Độ dài bước cố định** — tốc độ $O(1/k)$ (thuật toán ISTA);
- **Backtracking** — như GD nhưng kiểm tra trên toán tử gradient tổng quát $G_t(w)$;
- **Acceleration** — thêm động lượng Nesterov, tốc độ $O(1/k^2)$ (thuật toán FISTA [1]).

Nhận xét: đúng một mẹo động lượng biến GD thành AGD ($\kappa \rightarrow \sqrt{\kappa}$) cũng biến ISTA thành FISTA ($1/k \rightarrow 1/k^2$) — gia tốc là một *nguyên lý* chung.

7. MỞ RỘNG SANG BÀI TOÁN LASSO

7.4. Dò bước cho ISTA — ba vòng mới tới

→ Chọn $t = 7/L$



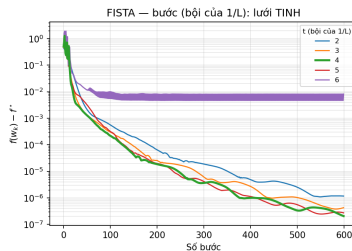
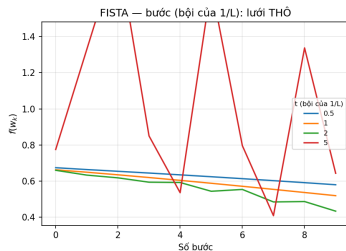
Nhận xét:

- Phải dò **ba vòng**: cả $2,5/L$ lẫn $7/L$ đều chạm mép trên, tới vòng thứ ba mới thấy $10/L$ hồng hẩn và $7/L$ là cực tiểu nội.
- Lý thuyết proximal đòi $t \leq 1/L$. Bước tốt nhất là $7/L$ — **cận bi quan gấp 7 lần**, đúng kiểu đã gặp ở GD (mục 4.7, bước 0,4 so với ngưỡng $2/L = 0,142$).
- Điểm chung cả bài: các cận $1/L, 2/L$ là đủ để có bảo đảm, không cần để chạy tốt. Nhưng **đừng đem $7/L$ này sang FISTA** — mục 7.5.

7. MỞ RỘNG SANG BÀI TOÁN LASSO

7.5. Dò bước cho FISTA — vì sao không dùng lại $7/L$ của ISTA

→ Chọn $t = 4/L$



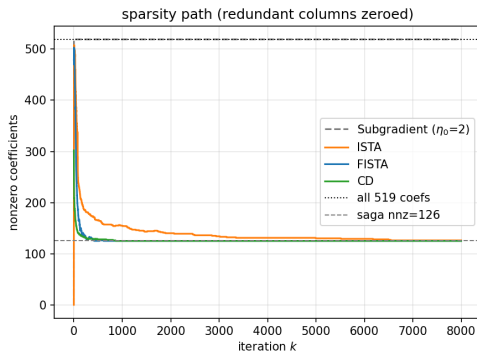
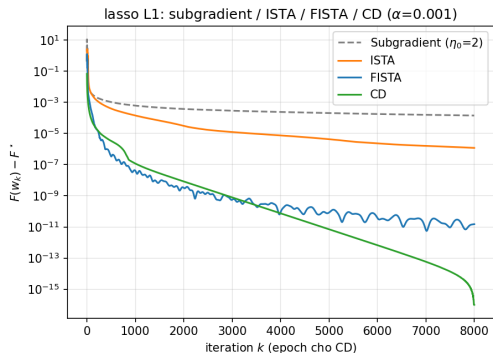
Nhận xét: lưới tinh 600 vòng: $2 \rightarrow 1,17 \times 10^{-6}$, $3 \rightarrow 4,26 \times 10^{-7}$, $4 \rightarrow 2,06 \times 10^{-7}$, $5 \rightarrow 2,72 \times 10^{-7}$, rồi 6

sập xuống $8,15 \times 10^{-3}$ — và cùng lúc số hệ số khác 0 vọt từ 125 lên 346, tức nghiệm mất luôn tính thưa.

- **Trần bước của FISTA thấp hơn của ISTA:** vách sập nằm giữa $5/L$ và $6/L$, trong khi ISTA vẫn chạy tốt ở $7/L$. Bản gia tốc chịu bước ngắn hơn bản thường.
- Cùng cơ chế với mục 4.12: momentum tích lũy phần vượt ngưỡng qua các vòng, còn ISTA thuần thì mỗi bước tự hấp thụ. Gia tốc mua tốc độ bằng *biên an toàn*.
- Bài học quy trình: **tham số dò cho thuật toán nào chỉ dùng cho thuật toán đó**. Thử dùng lại $7/L$ cho FISTA cho $1,17 \times 10^{-2}$ — tệ hơn cả bước $1/L$ theo sách.

7. MỞ RỘNG SANG BÀI TOÁN LASSO

7.6. Kết quả trên bộ dữ liệu Lasso ($d = 519$)



Nhận xét: bốn phương pháp, mỗi cái ở bước **dò tay** của riêng nó (mục 7.2, 7.4, 7.5). Thứ tự về đích đúng như tốc độ lý thuyết $O(1/\sqrt{k}) < O(1/k) < O(1/k^2)$: subgradient $1,4 \times 10^{-4}$, ISTA $1,2 \times 10^{-6}$, FISTA $1,5 \times 10^{-11}$, còn **CD** — solver scikit-learn thực dùng cho Lasso [4] — về đích f^* . Bảng số và chuyện tính thừa ở mục 7.7.

7. MỞ RỘNG SANG BÀI TOÁN LASSO

7.7. Lasso — số bước, độ chính xác và tính thưa

Nhận xét:

Thuật toán	Tốc độ LT	Số bước	Hệ số $\neq 0$
Subgradient	$O(1/\sqrt{k})$	8 000 [†]	519
ISTA	$O(1/k)$	8 000 [†]	126
FISTA	$O(1/k^2)$	8 000 [†]	125
CD	(thực tế nhanh)	8 000 [†]	125
saga	—	5 000	126

[†] chậm trên 8 000 vòng. Mỗi thuật toán ở bước đã dò tay:

subgradient $\eta_0=2$ (7.2), ISTA 7/L (7.4), FISTA 4/L (7.5). $f - f^*$

cuối: subgradient $1,4 \cdot 10^{-4}$, ISTA $1,2 \cdot 10^{-6}$, FISTA

$1,5 \cdot 10^{-11}$, **CD** 0, saga $7,1 \cdot 10^{-7}$.

- **Subgradient để lại đủ 519/519 hệ số khác 0** — không thừa chút nào; prox đưa xuống 125. Nó cũng chậm nhất, và thứ tự $O(1/\sqrt{k}) < O(1/k) < O(1/k^2)$ hiện ra đúng dù mỗi thuật toán đã được cho bước tốt nhất của riêng nó: $1,4 \times 10^{-4}$, $1,2 \times 10^{-6}$, $1,5 \times 10^{-11}$ — cách nhau đều đặn ~ 2 và ~ 5 bậc.

- $\text{sign}(w) = \pm 1$ ở mọi tọa độ khác 0 nên subgradient đẩy tọa độ qua 0 rồi đổi dấu, không bao giờ đáp đúng vào 0. Soft-thresholding không phải cách nhanh hơn để làm cùng việc — nó là phần tạo ra tính thưa.

- **CD** thắng tất cả (đạt đúng f^*) và là solver scikit-learn thực dùng cho Lasso [4]. Bốn cài đặt độc lập — ISTA, FISTA, CD, saga — cùng cho **125–126/519**: L1 loại $\sim 76\%$ cột. Việc dò bước cũng sửa luôn ISTA: ở 1/L nó còn để lại 148 hệ số, ở 7/L thì về 126 — cùng tập thưa với ba phương pháp kia.

8. KẾT LUẬN

8.1. Về dữ liệu và về từng thuật toán

- **Chuẩn hóa không phải mẹo tinh chỉnh mà là điều kiện để chạy được.** Ở $d = 415$, bộ thô có $\kappa \approx 10^{19}$ — vượt dài số thực 64 bit — nên λ thành vô hình, Hessian suy biến về mặt số học và **Newton không chạy nổi**; GD/AGD sau 2 000 bước còn cách nghiệm 0,434 (mục 4.1).
- Thực nghiệm khẳng định κ vs $\sqrt{\kappa}$: độ dốc log–log đo được là 0,819 (GD) và 0,413 (AGD), **tỉ số** $1,98 \approx 2$. Cả hai dưới giá trị lý thuyết vì $\kappa \log \frac{1}{\epsilon}$ là *cận trên* (mục 5.5).
- **Backtracking giúp GD nhưng phá AGD** (vòng chạy riêng, trần 5 000 bước): GD đi từ $3,3 \times 10^{-4}$ xuống $7,8 \times 10^{-8}$ — hơn **4 bậc**; còn AGD từ hội tụ sau 2 131 bước thành *không hội tụ*. Đổi bước mỗi vòng là phá cặp (t, β) ; đổi t mà *tính lại* β thì không sao — đó là cấu hình AGD ở mục 4.17, nhanh hơn bản $1/L$ 2,0 lần.
- **Newton nhanh nhất trong bảy cấu hình của nhóm, trên cả hai trục:** 8–10 bước so với hàng nghìn, và 6,0s so với 11,4–583,2s (mục 4.18). Nhưng nó **thua quasi-Newton của thư viện**: chi phí $O(nd^2 + d^3)$ đủ nặng để lbfgs về trước (3,31s so với 10,21s, mục 4.19) — đúng điểm hòa vốn dự báo ($d^* \approx 139$).
- **Cài đặt:** giải hệ bằng Cholesky, không nghịch đảo Hessian. Ma trận phải **đủ hạng** — ở $d \sim 400$ điều đó không tự nhiên mà có: corr/VIF không bắt được cộng tuyến *chính xác*, phải thêm QR xoay cột.
- **SGD** đạt $1,6 \times 10^{-3}$ sau 50 epoch rồi dừng ở sàn phương sai.

8. KẾT LUẬN

8.2. Về các phần mở rộng và về độ tin cậy của cài đặt

- Họ **bước thích nghi** *không* thắng nổi GD một khi GD được dò bước tử tế: chỉ Adam (1,3 bậc) và AMSGrad (3,2 bậc) vượt lên, còn AdaGrad, RMSprop, AdamW đều **thua** GD ở $t = 0,4$ (mục 6.7). Và không cái nào thắng AGD — ở $d = 415$ chỉ AGD và Newton đạt tiêu chí dừng. Đáng chú ý, ở $d = 39$ Adam từng *thắng* AGD (413 so với 800 bước): cùng đoạn code, bài toán lớn hơn thì kết luận **đảo chiều**.
- Với bài toán **không khả vi** (Lasso), nhóm đã chạy cả subgradient để đối chứng: nó để lại 519/519 hệ số khác 0 trong khi proximal đưa xuống 125, và còn chậm hơn cả ISTA. Soft-thresholding không phải cách nhanh hơn để làm cùng một việc — nó là phần tạo ra tính thừa.
- Cùng **mẹo động lượng** biến GD thành AGD ($\kappa \rightarrow \sqrt{\kappa}$) cũng biến ISTA thành FISTA ($1/k \rightarrow 1/k^2$): gia tốc là một *nguyên lý* chung.
- Kết quả code nhóm tự cài **khớp với thư viện** scikit-learn tới độ chính xác máy ($|f - f^*| \sim 10^{-17}$), và ba cài đặt L1 độc lập (FISTA, CD, saga) cùng hội tụ về **125–126 / 519** hệ số khác 0 — hai phép kiểm tra chéo độc lập cho thấy cài đặt là đúng.
- f^* kèm **chứng chỉ** chứ không đòi tin: $\|\nabla f(w^*)\| = 7,9 \times 10^{-15}$ và tính lồi mạnh cho $f^* - f_{\text{true}} \leq 3,1 \times 10^{-26}$ (mục 4.2, Phụ lục B).

Tài liệu tham khảo I

- [1] Amir Beck and Marc Teboulle. “A Fast Iterative Shrinkage-Thresholding Algorithm for Linear Inverse Problems”. In: *SIAM Journal on Imaging Sciences* 2.1 (2009), pp. 183–202.
- [2] Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Ch. 9: descent methods, line search, Newton. Cambridge University Press, 2004.
- [3] John Duchi, Elad Hazan, and Yoram Singer. “Adaptive Subgradient Methods for Online Learning and Stochastic Optimization”. In: *Journal of Machine Learning Research* 12 (2011), pp. 2121–2159.
- [4] Jerome Friedman, Trevor Hastie, and Robert Tibshirani. “Regularization Paths for Generalized Linear Models via Coordinate Descent”. In: *Journal of Statistical Software* 33.1 (2010), pp. 1–22.
- [5] Diederik P. Kingma and Jimmy Ba. “Adam: A Method for Stochastic Optimization”. In: *International Conference on Learning Representations (ICLR)*. 2015.

Tài liệu tham khảo II

- [6] Ilya Loshchilov and Frank Hutter. “Decoupled Weight Decay Regularization”. In: *International Conference on Learning Representations (ICLR)*. 2019.
- [7] Yuri Nesterov. *Lectures on Convex Optimization*. 2nd ed. Springer, 2018.
- [8] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. 2nd ed. Ch. 3 line search, Ch. 6 quasi-Newton. Springer, 2006.
- [9] Brendan O’Donoghue and Emmanuel Candès. “Adaptive Restart for Accelerated Gradient Schemes”. In: *Foundations of Computational Mathematics* 15.3 (2015), pp. 715–732.
- [10] Pedregosa et al. *scikit-learn: LogisticRegression*. solver lbfgs / saga; objective form & C parameter. 2011.
- [11] Sashank J. Reddi, Satyen Kale, and Sanjiv Kumar. “On the Convergence of Adam and Beyond”. In: *International Conference on Learning Representations (ICLR)*. 2018.

Tài liệu tham khảo III

- [12] Herbert Robbins and Sutton Monro. “A Stochastic Approximation Method”.
In: *The Annals of Mathematical Statistics* 22.3 (1951), pp. 400–407.

Phụ lục

Phụ lục A

Cài đặt lỗi và kiểm chứng đạo hàm

```
1 # reg[j] = 1, RIENG reg[-1] = 0: he so chan KHONG bi phat (muc 5.1)
2 def value(w, X, y, lam, reg):          # on dinh so: dung logaddexp
3     z = X @ w
4     return np.mean(np.logaddexp(0.0, z) - y*z) + 0.5*lam*np.sum((reg*w)**2)
5
6 def grad(w, X, y, lam, reg):
7     p = sigmoid(X @ w)
8     return X.T @ (p - y) / len(y) + lam*reg*w
9
10 def hessian(w, X, y, lam, reg):        # Newton = IRLS
11     p = sigmoid(X @ w); s = p*(1-p)
12     H = (X.T * s) @ X / len(y)
13     H[np.diag_indices(len(w))] += lam*reg
14     return H
```

- Kiểm chứng: `scipy.optimize.check_grad` cho sai số $\approx 10^{-7}$; Hessian đối chiếu bằng sai phân trung tâm, khớp $\sim 10^{-7}$.
- Newton dùng `cholesky + solve`, không dùng `inv`.
- `reg` không phải chi tiết vụn: chính số 0 ở hệ số chặn làm $\mu \neq \lambda$, và là lý do mục 4.2 phải đo μ^* thay vì thay λ vào.

Phụ lục B

Giao thức đo và chỉ số phân loại

Giao thức đo:

- Khởi tạo chung $w_0 = 0$; dừng khi $\|\nabla f\|_2 \leq 10^{-10}$ hoặc chạm trần số bước. Bảng 4.19 dùng trần chặt hơn (10^{-11} ; 10^{-12} cho Newton), nên số vòng ở đó lớn hơn ở mục 4.17.
- f^* là giá trị **tại điểm cuối** của Newton damped chạy sâu ($\|\nabla f\|_2 < 10^{-14}$, mục 4.2), *không* lấy min cả lịch sử. Bảo chứng là bất đẳng thức $f(w) - f^* \leq \|\nabla f(w)\|^2 / (2\mu_*)$ với $\mu_* = \lambda_{\min}(\nabla^2 f(w^*)) = 1,00 \times 10^{-3}$ *đo tại nghiệm* (không phải λ theo nguyên tắc — mục 5.1). Đo được: $\|\nabla f(w^*)\| = 7,9 \times 10^{-15}$ cho cận $3,1 \times 10^{-26}$, tức thấp hơn sàn 10^{-16} của mọi đồ thị **mười bậc**.
- Bài L1 không lồi mạnh $\Rightarrow$ không có cận trên; ở đó f^* là giá trị nhỏ nhất mà *mọi* lần chạy đạt được — hướng an toàn, chỉ làm khoảng cách báo cáo lớn hơn.
- AGD ghi $f(w_k)$ cùng $\|\nabla f(y_{k-1})\|$ — cùng hàng, cùng điểm: bổ đề giảm với bước $1/L$ cho $f(w_k) \leq f(y_{k-1})$, nên tiêu chí dừng chứng nhận đúng đại lượng được vẽ.
- Thời gian: perf_counter, làm nóng 1 vòng, trung vị 3–5 lần. BLAS ghim 1 luồng và được **kiểm tra lại** qua threadpoolctl lúc chạy.
- Môi trường: Apple M1, 1 luồng BLAS; Python 3.12, numpy 1.26, scikit-learn 1.5.

Phụ lục C

Chỉ số phân loại (ngoài trọng tâm)

Chỉ số phân loại (*in-sample*, trên toàn bộ $n = 75\,026$ dòng — chỉ để tham khảo, không phải trọng tâm của bài; pipeline không chia train/test vì đây là bài toán tối ưu):

	Tỉ lệ rời bỏ	AUC	Accuracy	F1	Precision/Recall
ridge	10,8%	0,910	0,912	0,494	0,647/0,399

CẢM ƠN THẦY VÀ CÁC BẠN ĐÃ LẮNG NGHE